

1. Introduction

Most snippets assume the following macros.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define P push_back
6 #define I insert
7 #define X extract
8 #define L lower_bound
9 #define U upper_bound
10 #define C count
11
12 auto ckmn = [](auto &x, auto y) { return y < x ? x = y, true : false; };
13 auto ckmx = [](auto &x, auto y) { return y > x ? x = y, true : false; };
14 auto lso = [](integral auto n) { return n & (-n); };
15 auto pop = [](integral auto n) { return popcount((unsigned long long)n); };
16
17 #define W(x, y) ((x & y) == y)
18 #define N(i, n) " \n"[i == n]
19 #define Z(x, i) (x >> i & 1)
20 #define F(i, l, r) for (int i = l; i < r; i++)
21 #define G(i, l, r) for (int i = l; i <= r; i++)
22 #define H(i, r, l) for (int i = r; i >= l; i--)
23 #define A(a) (a).begin(), (a).end()
24 #define R(a) (a).rbegin(), (a).rend()
25 #define S(a) (int)ssize(a)
26 #define B(a) (a).begin()
27 #define E(a) (a).end()
28 #define T(a) prev(E(a))
29 #define O(a) sort(A(a)), a.erase(unique(A(a)), E(a))
30
31 template <typename T = int>
32 using V = vector<T>;
33 using ll = long long;
34 using ull = unsigned long long;
35 using ii = array<int, 2>;
36
37 constexpr int oo = 0x3f3f3f3f;
38 constexpr ll OO = 0x3f3f3f3f3f3f3f3f;
```

1.1. Template (Proton)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 typedef vector<ll> vll;
5 typedef vector<vll> vvll;
6 typedef pair<ll, ll> pll;
7 typedef vector<pll> vpll;
8 typedef vector<int> vi;
9 typedef vector<vi> vvi;
10 typedef pair<int, int> pi;
11 typedef vector<pi> vpi;
12 typedef set<ll> sll;
13 typedef set<int> si;
14 typedef map<ll, ll> mll;
15 typedef map<int, int> mi;
16 typedef vector<bool> vb;
17 #define pb push_back
18 #define INF(dt) numeric_limits<dt>::max()
19 #define NINF(dt) numeric_limits<dt>::min()
```

1.2. Template (Baytoro)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long
4 #define pb push_back
5 #define fr first
6 #define sc second
7 #define all(x) (x).begin(), (x).end()
8 #define rall(x) (x).rbegin(), (x).rend()
9 #define int ll
```

1.3. Compilation

For convenience, in the .bashrc, you may want to define

```
alias g='compile_and_run_cpp'

where compile_and_run_cpp is defined as:

compile_and_run_cpp() {
    g++ -fsanitize=undefined -std=c++23 -Werror=uninitialized -Winvalid-pch -Wall -Wextra -o out "$1" && ./out
}
```

1.4. I/O

For large inputs, it may be necessary to enable fast I/O.

```
cin.tie(nullptr)->sync_with_stdio(false);
```

Regarding interactive problems, remember to flush the stream using endl!

1.5. Order Statistic Tree

```
1 #include <bits/extc++.h>
2
3 using namespace std;
4
5 using Ost = __gnu_pbds::tree<
6     int, __gnu_pbds::null_type, less<>, __gnu_pbds::rb_tree_tag,
7     __gnu_pbds::tree_order_statistics_node_update>;
```

1.6. Custom Hash

While this slows down unordered_map a little, it helps prevent hacks.

```
1 struct custom_hash { // PERF: https://codeforces.com/blog/entry/62393
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x^(x>>30)) * 0xbf58476d1ce4e5b9;
5         x = (x^(x>>27)) * 0x94d049bb133111eb;
6         return x^(x>>31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t FIXED_RANDOM
10             = chrono::steady_clock::now().time_since_epoch().count();
11         return splitmix64(x + FIXED_RANDOM);
12     }
13 };
```

1.7. Random Number Generation

To generate a random integer, first set up gen.

```
mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
```

After which, you can use uniform_int_distribution to produce a random integer in a closed interval.

2. Disjoint Set Union

Nodes with negative values are roots. cmpCnt and cmpMax find the number of components and the size of the largest component respectively. Omit if not necessary.

```
1 struct DSU {
2     int cmpCnt, cmpMax;
3     V<int> nodes; // WARN: assumes `nodes` is zero-indexed
4     DSU(int n)
5         : cmpCnt(n)
6         , cmpMax(1)
7         , nodes(n, -1) {}
8     int find(int x) { return nodes[x] < 0 ? x : nodes[x] = find(nodes[x]); }
9     int size(int x) { return -nodes[find(x)]; }
10    bool unite(int x, int y) { // WARN: merge `y` into `x`
11        x = find(x), y = find(y);
```

```
12    if (x == y) return false;
13    if (nodes[x] > nodes[y]) swap(x, y);
14    nodes[x] += nodes[y], nodes[y] = x;
15    cmpCnt--;
16    ckmx(cmpMax, -nodes[x]);
17    return true;
18 }
19 };
```

2.1. Persistent Variant

Check if two nodes are connected at some point in time.

```
1 struct DSU {
2     V<int> size, parent, time_changed; // WARN: assumes `parent` is zero-indexed
3     DSU(int n)
4         : size(n, 1)
5         , parent(n, 0)
6         , time_changed(n, 0) {
7         iota(A(parent), 0);
8     }
9     int find(int x, int time) {
10        if (parent[x] == x || time_changed[x] > time) return x;
11        return find(parent[x], time);
12    }
13    bool unite(int x, int y, int time) { // WARN: merge `y` into `x`
14        x = find(x, time), y = find(y, time);
15        if (x == y) return false;
16        if (size[x] < size[y]) swap(x, y);
17        size[x] += size[y], parent[y] = x, time_changed[y] = time;
18        return true;
19    }
20 };
```

2.2. Kruskal Reconstruction Tree

This data structure stores the implicit tree formed across unite operations. kweight stores the weight of the edge that was used in unite. ops is the total number of operations. Even if x and y are connected, we would just add a single edge in the KRT. The eventual root will be kidx, which you may use to run the DFS to solve.

```
1 struct DSU {
2     int kidx;
3     vector<int> nodes, krep, kweight; // WARN: assumes `nodes` is zero-indexed
4     vector<vector<int>> krt;
5     DSU(int n, int ops)
6         : kidx(n - 1)
7         , nodes(n, -1)
8         , krep(n + ops, 0)
9         , kweight(n + ops, 0)
10        , krt(n + ops) {
11        iota(A(krep), 0);
12    }
13    int find(int x) { return nodes[x] < 0 ? x : nodes[x] = find(nodes[x]); }
14    int size(int x) { return -nodes[find(x)]; }
15    bool unite(int x, int y, int w) { // WARN: merge `y` into `x`
16        x = find(x), y = find(y);
17        if (x == y) {
18            kidx++;
19            krt[kidx].P(krep[x]);
20            kweight[kidx] = w;
21            krep[x] = kidx;
22            return false;
23        }
24        if (nodes[x] > nodes[y]) swap(x, y);
25        nodes[x] += nodes[y], nodes[y] = x;
26        kidx++;
27        krt[kidx].P(krep[x]);
28        krt[kidx].P(krep[y]);
29        kweight[kidx] = w;
30        krep[x] = kidx;
31        return true;
32    }
33    void solve(int u, int prev_w = oo) {
34        int min_w = min(prev_w, kweight[u]);
35        for (auto v : krt[u]) {
```

```
36     solve(v, min_w);
37 }
38 }
39 };
```

3. Sorting

3.1. Custom struct/class

Suppose we have some custom struct representing fractions, to create a set, we need to perform operator overloading of <.

```
1 struct Frac {
2     int x, y;
3     Frac(int x, int y)
4         : x(x)
5         , y(y) {}
6     bool operator<(Frac other) const { return x * other.y < y * other.x; }
7 };
```

3.2. Inversions Count

An array's *inversions count* refers to the number of adjacent swaps required to sort an array. It can be computed via a modified merge sort, which has the side effect of sorting the array.

```
1 auto merge = [](auto self, V<int> &v, int lb, int rb) -> ll {
2     if (lb == rb) return 0;
3     int mb = (lb + rb) / 2, lp = lb, rp = mb + 1;
4     ll ans = self(self, v, lb, mb) + self(self, v, mb + 1, rb);
5     while (lp <= mb && rp <= rb) {
6         if (v[lp] <= v[rp]) lp++;
7         else ans += mb - lp + 1, rp++;
8     }
9     inplace_merge(B(v) + lb, B(v) + mb + 1, B(v) + rb + 1);
10    return ans;
11 };
```

4. Matching

4.1. Maximum Cardinality Bipartite Matching (MCMF)

A maximum matching contains as many edges of a graph G as possible such that each vertex is part of at most one edge of the subset. Assume that $0 \leq u < lsize$ and $0 \leq v < rsize$ in the following implementation.

```
1 int l_size, r_size, edge_cnt;
2 mt19937 gen(chrono::steady_clock::now().time_since_epoch().count());
3
4 int main() {
5     cin >> l_size >> r_size >> edge_cnt;
6     V<ii> edg, matching;
7
8     // WARN: assumes both `u` and `v` are zero-indexed
9     for (int i = 0, u, v; i < edge_cnt; i++) {
10         cin >> u >> v;
11         edg.push_back({ u, v });
12     }
13
14     shuffle(A(edg), gen);
15
16     auto hopcroft_karp = [&]() -> void {
17         V<int> deg(l_size + 1);
18         for (auto &[u, v] : edg) deg[u]++;
19         partial_sum(begin(deg), end(deg), begin(deg));
20         V<int> g(edge_cnt), lmc(l_size, -1), rmc(r_size, -1), a, p, q(l_size);
21         for (auto &[u, v] : edg) g[--deg[u]] = v;
22         while (true) {
23             a.assign(l_size, -1), p.assign(l_size, -1);
24             int t = 0, match = false;
25             for (int i = 0; i < l_size; i++) {
26                 if (lmc[i] == -1) q[t++] = a[i] = p[i] = i;
```

```
27     }
28     for (int i = 0; i < t; i++) {
29         int x = q[i];
30         if (!lmc[a[x]]) continue;
31         for (int j = deg[x]; j < deg[x + 1]; j++) {
32             int y = g[j];
33             if (rmc[y] == -1) {
34                 while (-y) {
35                     rmc[y] = x, swap(lmc[x], y), x = p[x];
36                 }
37                 match = true;
38                 break;
39             }
40             if (p[rmc[y]] == -1) {
41                 q[t++] = y = rmc[y], p[y] = x, a[y] = a[x];
42             }
43         }
44     }
45     if (!match) break;
46 }
47 for (int i = 0; i < l_size; i++) {
48     if (!lmc[i]) matching.push_back({ i, lmc[i] });
49 }
50 }
51 };
```

4.1.1. Minimum Vertex Cover

Assuming you have obtained the matching above, you can retrieve the *minimum vertex cover* (the smallest set of vertices that includes at least one endpoint of every edge of the graph) by constructing a new *directed* graph. Edges that belong to the matching will go from right to left, whereas all other edges will go from left to right. Perform a depth first search starting at all left vertices that are not incident to any edges in the matching.

The minimum vertex cover includes all visited right vertices of the matching, as well as all unvisited left vertices of the matching.

Note that some problems also have situations where there are isolated nodes (degree zero) that have to be taken.

```
1 int l_size, r_size;
2 V<ii> matching, edg;
3 V<int> lhs, rhs;
4
5 int main() {
6     set<ii> seen;
7     set<ii> matching_set(A(matching));
8
9     map<ii, V<ii>> adj;
10    set<int> to_start;
11    V<int> min_cover;
12
13    F(i, 0, l_size) to_start.I(i);
14
15    for (auto [u, v] : edg) {
16        if (matching_set.C({ u, v })) {
17            adj[{ v, 1 }].P({ u, 0 });
18            to_start.X(u);
19        } else {
20            adj[{ u, 0 }].P({ v, 1 });
21        }
22    }
23
24    auto dfs = [&](auto self, int u, int state) -> void {
25        seen.I({ u, state });
26        for (auto [v, nstate] : adj[{ u, state }]) {
27            if (seen.C({ v, nstate })) continue;
28            self(self, v, nstate);
29        }
30    };
31
32    for (auto u : to_start) {
33        if (seen.C({ u, 0 })) continue;
34        dfs(dfs, u, 0);
```

```
35 }
36
37 for (auto [u, v] : matching) {
38     if (seen.C({ v, 1 })) {
39         min_cover.P(rhs[v]);
40     }
41     if (!seen.C({ u, 0 })) {
42         min_cover.P(lhs[u]);
43     }
44 }
45 };
```

4.2. Perfect Matching

A perfect matching is one where every vertex is included in exactly one edge of the matching. The following implementation finds the minimum weight perfect matching. To obtain the maximum weight variant, simply invert all the costs.

```
1 template <typename T>
2 class Hungarian {
3     const T INF = numeric_limits<T>::max();
4
5 public:
6     T tot = 0;
7     vector<int> p, way, ans;
8     vector<T> u, v;
9     Hungarian(const vector<vector<T>> &a) { // WARN: assumes `a` is one-indexed
10         int n = ssize(a) - 1, m = ssize(a[0]) - 1;
11         u.assign(n + 1, 0), v.assign(m + 1, 0);
12         p.assign(m + 1, 0), way.assign(m + 1, 0), ans.assign(n + 1, 0);
13         for (int i = 1; i <= n; i++) {
14             p[0] = i;
15             int j0 = 0;
16             vector<bool> usd(m + 1);
17             vector<T> mnv(m + 1, INF);
18             do {
19                 usd[j0] = true;
20                 int i0 = p[j0], j1 = 0;
21                 T dlt = INF;
22                 for (int j = 1; j <= m; j++) {
23                     if (!usd[j]) {
24                         T cur = a[i0][j] - u[i0] - v[j];
25                         if (cur < mnv[j]) mnv[j] = cur, way[j] = j0;
26                         if (mnv[j] < dlt) dlt = mnv[j], j1 = j;
27                     }
28                 }
29                 for (int j = 0; j <= m; j++) {
30                     if (usd[j]) u[p[j]] += dlt, v[j] -= dlt;
31                     else mnv[j] -= dlt;
32                 }
33                 j0 = j1;
34             } while (p[j0]);
35             do {
36                 int j1 = way[j0];
37                 p[j0] = p[j1];
38                 j0 = j1;
39             } while (j0);
40         }
41         for (int j = 1; j <= m; j++) {
42             ans[p[j]] = j;
43         }
44         tot = -v[0];
45     };
46 };
```

5. Square Root Decomposition

Assuming block represents the size of a chunk, a represents the original, full-sized array, and b represents the values of each block, the example snippet demonstrates a sum query. A good value for block is around 250.

```
1 // WARN: assumes zero-indexed queries
2 auto query = [](int l, int r) -> ll {
3     ll ans = 0;
4     int bl = l / block, br = r / block;
```

```
5 if (bl == br) return reduce(begin(a) + l, begin(a) + r + 1);
6 for (int i = l; i <= (bl + 1) * block - 1; i++) ans += a[i];
7 for (int i = bl + 1; i <= br - 1; i++) ans += b[i];
8 for (int i = br * block; i <= r; i++) ans += a[i];
9 return ans;
10};
```

5.1. Mo's Algorithm

```
1 // WARN: assumes zero-indexed queries
2 struct Query {
3     int i, l, r;
4     Query(int i, int l, int r)
5         : i(i)
6         , l(l)
7         , r(r) {}
8     friend bool operator<(const Query &qa, const Query &qb) {
9         if (qa.l / block == qb.l / block) return qa.r < qb.r;
10        return qa.l < qb.l;
11    }
12};
13V<Query> queries;
14
15void solve() {
16    auto ins = [&](int i) {};
17    auto del = [&](int i) {};
18
19    int cl = 0, cr = -1, cur = 0;
20
21    for (auto [i, l, r] : queries) {
22        while (cl > l) ins(--cl);
23        while (cr < r) ins(++cr);
24        while (cl < l) del(cl++);
25        while (cr > r) del(cr--);
26        ans[i] = cur;
27    }
28}
```

6. Treap

6.1. Implicit Variant

Insertion and deletion are done based on the zero-indexed position of an element within the tree. Nodes in the snippet store string values, remember to modify depending on the problem.

```
1 mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
2 uniform_int_distribution dist(0, oo);
3
4 struct Node {
5     string val;
6     int pri, size;
7     Node *lp, *rp;
8     Node(string val)
9         : val(val)
10        , pri(dist(gen))
11        , size(1)
12        , lp(nullptr)
13        , rp(nullptr) {}
14 };
15
16 int size(Node *root) { return root ? root->size : 0; }
17
18 void split(Node *root, Node *&lt, Node *&rt, int idx) {
19     if (!root) {
20         lt = rt = nullptr;
21         return;
22     }
23     if (size(root->lp) + 1 <= idx) {
24         split(root->rp, root->rp, rt, idx - (size(root->lp) + 1));
25         lt = root;
26     } else {
27         split(root->lp, lt, root->lp, idx);
28         rt = root;
29     }
30     root->size = 1 + size(root->lp) + size(root->rp);
31 }
```

```
31 }
32
33 void merge(Node *&root, Node *lt, Node *rt) {
34     if (!lt || !rt) {
35         root = lt ? lt : rt;
36         return;
37     }
38     if (lt->pri > rt->pri) {
39         merge(lt->rp, lt->rp, rt);
40         root = lt;
41     } else {
42         merge(rt->lp, lt, rt->lp);
43         root = rt;
44     }
45     root->size = 1 + size(root->lp) + size(root->rp);
46 }
47
48 void concat(Node *root, vector<string> &ans) {
49     if (!root) return;
50     concat(root->lp, ans);
51     ans.push_back(root->val);
52     concat(root->rp, ans);
53 }
54
55 void clean(Node *root) {
56     if (!root) return;
57     clean(root->lp);
58     clean(root->rp);
59     delete root;
60 }
61
62 int n, q, find_idx = -1;
63
64 void find(Node *root, string &s, int offset) {
65     if (!root) return;
66
67     if (root->val == s) {
68         int cur = offset;
69
70         if (root->lp) cur += root->lp->size;
71         find_idx = cur;
72     }
73     return;
74 }
75
76 int inc = 1;
77 if (root->lp) inc += root->lp->size;
78
79 find(root->lp, s, offset);
80 find(root->rp, s, offset + inc);
81 }
82
83 void solve_t() {
84     cin >> n >> q;
85
86     vector<string> a(n);
87
88     F(i, 0, n) cin >> a[i];
89
90     Node *root = new Node(a[0]);
91
92     auto insert = [&](int i, string s) -> void {
93         Node *lp, *rp;
94         split(root, lp, rp, i);
95         Node *n_node = new Node(s);
96         merge(lp, lp, n_node);
97         merge(lp, lp, rp);
98         root = lp;
99     };
100
101     auto erase = [&](int i) -> void {
102         Node *lp, *rp, *n_lp, *n_rp;
103
104         split(root, lp, rp, i);
105         split(rp, n_lp, n_rp, 1);
106
107         V<string> ans;
108
109         merge(lp, lp, n_rp);
110         root = lp;
111     }
```

```
111 };
112
113 clean(root);
114 }
```

6.2. Cartesian Tree

We may also construct it from an array, assuming no further updates to the array are required. The smallest value belongs to the `root`, which stores the corresponding index. Assuming all values are distinct, the resulting tree is unique.

```
1 template <typename T>
2 class CartesianTree {
3     struct Node {
4         int p, l, r;
5         Node(int p = 0, int l = 0, int r = 0)
6             : p(p)
7             , l(l)
8             , r(r) {}
9     };
10
11 public:
12     int root;
13     deque<Node> tree;
14     CartesianTree(const vector<T> &a)
15         : tree(S(a)) {
16         for (int i = 1; i <= S(a) - 1; i++) { // WARN: `a` is one-indexed
17             tree[i].p = i - 1;
18             while (a[i] < a[tree[i].p]) tree[i].p = tree[tree[i].p].p;
19             tree[i].l = tree[tree[i].p].r;
20             tree[tree[i].l].p = i;
21             tree[tree[i].p].r = i;
22         }
23         for (int i = 1; i <= S(a) - 1; i++)
24             if (tree[i].p == 0) root = i;
25     }
26 };
```

6.3. Lazy Propagation

The following snippet illustrates a non-implicit treap with range addition.

```
1 mt19937 gen(chrono::steady_clock::now().time_since_epoch().count());
2 uniform_int_distribution dist(0, INT_MAX);
3
4 struct Node {
5     int val, pri, add, q; // WARN: rmb to tweak (may not need `q`)
6     Node *lp, *rp;
7     Node(int val, int q)
8         : val(val)
9         , pri(dist(gen))
10        , add(0)
11        , q(q)
12        , lp(0)
13        , rp(0) {}
14 };
15
16 void push(Node *root) {
17     if (!root) return;
18     if (root->add) {
19         if (root->lp) root->lp->val += root->add, root->lp->add += root->add;
20         if (root->rp) root->rp->val += root->add, root->rp->add += root->add;
21         root->add = 0;
22     }
23 }
24
25 void split(Node *root, Node *&lt, Node *&rt, int x) {
26     if (!root) {
27         lt = rt = nullptr;
28         return;
29     }
30     push(root), push(lt), push(rt);
31     if (root->val <= x) {
32         split(root->rp, root->rp, rt, x);
33         lt = root;
```

```
34 } else {
35     split(root->lp, lt, root->lp, x);
36     rt = root;
37 }
38 }
39
40 void merge(Node *&root, Node *lt, Node *rt) {
41     push(root), push(lt), push(rt);
42     if (lt || !rt) {
43         root = lt ? lt : rt;
44         return;
45     }
46     if (lt->pri >= rt->pri) {
47         merge(lt->rp, lt->rp, rt);
48         root = lt;
49     } else {
50         merge(rt->lp, lt, rt->lp);
51         root = rt;
52     }
53 }
54
55 void range_add(Node *&root, int l, int r, int addend) {
56     Node *lt = nullptr, *lm = nullptr, *mt = nullptr, *rt = nullptr;
57     split(root, lm, rt, r);
58     split(lm, lt, mt, l - 1);
59     if (mt) mt->val += addend, mt->add += addend;
60     merge(lm, lt, mt);
61     merge(root, lm, rt);
62 }
63
64 void insert(Node *&root, int x, int q) {
65     push(root);
66     Node *lt = nullptr, *rt = nullptr;
67     split(root, lt, rt, x);
68     merge(lt, lt, new Node(x, q));
69     merge(root, lt, rt);
70 }
71
72 void concat(Node *&root, vector<ii> &a) {
73     if (!root) return;
74     push(root);
75     concat(root->lp, a);
76     a.push_back({ root->q, root->val });
77     concat(root->rp, a);
78 }
79
80 void clean(Node *root) {
81     if (!root) return;
82     clean(root->lp);
83     clean(root->rp);
84     delete root;
85 }
```

7. Graphs

7.1. Cycle Retrieval

In problems where $a[i]$ represents the next “jump” from position i , the following snippet finds all cycles, which may be used for further processing.

```
1 int main() {
2     V seen(n + 1), a(n + 1);
3
4     G(i, 1, n) cin >> a[i];
5
6     // WARN: assumes one-indexed `a`
7     for (int i = 1, j, cys; i <= n; i++) {
8         if (seen[i]) continue;
9         j = i, cys = 0;
10        set<int> curs;
11        deque<int> path;
12        do {
13            seen[j] = true, curs.insert(j), path.push_back(j);
14            j = a[j];
15            if (curs.contains(j)) {
16                cys = j;
17                break;
18            }
19        } while (j != i);
20        if (cys) {
21            // cycle found
22            // ...
23        }
24    }
25 }
```

```
18 }
19 } while (!seen[j]);
20 if (cys) {
21     while (path.front() != cys) path.pop_front();
22 }
23 }
24 }
```

7.2. Diameter

The furthest distance a node u is from some other node in a tree is equivalent to $\max(\text{dist}(\text{dia}_u, u), \text{dist}(\text{dia}_v, v))$, where dia_u and dia_v represent the endpoints of some diameter.

```
1 // WARN: assumes zero-indexed `adj`
2 auto build = [](vector<vector<int>> &adj) -> pair<int, vector<int>> {
3     int init = 0, n = S(adj);
4     V<int> df(n, 0), du(n, 0), dv(n, 0);
5
6     auto dfs
7         = [&](auto &&self, int u, int pu, vector<int> &d, int cur = 0) -> void {
8         d[u] = cur;
9         for (auto v : adj[u]) {
10             if (v == pu) continue;
11             self(self, v, u, d, cur + 1);
12         }
13     };
14
15     dfs(dfs, init, -1, df);
16     int dia_u = max_element(A(df)) - begin(df);
17
18     dfs(dfs, dia_u, -1, du);
19     int dia_v = max_element(A(du)) - begin(du);
20
21     dfs(dfs, dia_v, -1, dv);
22     vector<int> ans(n, 0);
23
24     F(i, 0, n) { ckmx(ans[i], max(du[i], dv[i])); }
25
26     return { *max_element(A(du)), ans };
27 };
```

7.3. Bridges

A *bridge* is an edge of a graph whose deletion increases the graph’s number of connected components. $\text{memo}[u]$ represents the number of *back-edges* passing over the edge between u and its parent.

```
1 int n;
2
3 int main() {
4     cin >> n;
5
6     V adj(n, V());
7     vector<ii> bridges;
8     vector<int> depth(n), memo(n);
9
10    // NOTE: assumes zero-indexed nodes
11    auto solve = [&](auto self, int u, int pu) -> void {
12        for (auto &v : adj[u])
13            if (v != pu) {
14                if (depth[v]) {
15                    depth[v] = depth[u] + 1;
16                    self(self, v, u);
17                    memo[u] += memo[v];
18                } else if (depth[v] < depth[u]) {
19                    memo[u]++;
20                } else {
21                    memo[u]--;
22                }
23            }
24        if (depth[u] != 1 && !memo[u]) bridges.push_back({ pu, u });
25    };
26
27    depth[0] = 1;
28    solve(solve, 0, -1);
29 }
```

```
29 }
```

7.4. Cut Vertices

When a *cut vertex* is removed, the graph is disconnected.

```
1 int n;
2
3 int main() {
4     cin >> n;
5
6     V adj(n, V());
7     vector<int> depth(n), memo(n, INT_MAX), cuts(n, false);
8
9     // NOTE: assumes zero-indexed nodes
10    auto solve = [&](auto self, int u, int pu) -> void {
11        int children = 0;
12        for (auto &v : adj[u])
13            if (v != pu) {
14                if (depth[v]) {
15                    depth[v] = depth[u] + 1;
16                    self(self, v, u);
17                    memo[u] = min(memo[u], memo[v]);
18                    children++;
19                } else if (depth[v] < depth[u]) {
20                    memo[u] = min(memo[u], depth[v]);
21                } else {
22                    memo[v] = min(memo[v], depth[u]);
23                }
24            }
25        if (pu == -1) {
26            cuts[u] = (children >= 2);
27        } else {
28            if (memo[u] >= depth[pu]) cuts[pu] = true;
29        }
30    };
31
32    depth[0] = 1;
33    solve(solve, 0, -1);
34 }
```

7.5. Edge Cactus

Every edge belongs to at most one cycle. You may find all cycles by merging the edges with DSU.

```
1 int n, m;
2
3 int main() {
4     cin >> n >> m;
5
6     V adj(n + 1, V<ii>());
7     V<ii> edg;
8
9     F(i, 0, m) {
10        int u, v;
11        cin >> u >> v;
12        adj[u].P({ v, i });
13        adj[v].P({ u, i });
14        edg.P({ u, v });
15    };
16
17    V mnb(n + 1, n + 1), depth(n + 1, 0);
18    mnb[n] = n;
19    DSU dsu(m);
20
21    auto solve = [&](auto &&self, int u, int pu, int ei) -> ii {
22        depth[u] = depth[pu] + 1;
23        ii dmn = { depth[u], ei };
24        for (auto [v, en] : adj[u]) {
25            if (v == pu) continue;
26            if (depth[v]) ckmn(dmn, ii{ depth[v], en });
27            else ckmn(dmn, (ii){ self(self, v, u, en) });
28        }
29        if (dmn[0] != depth[u]) {
30            int en = dmn[1];
31            dsu.unite(ei, en);
32        }
33    };
34 }
```

```

32 }
33 return dmn;
34 };
35
36 G(i, 1, n) {
37   if (depth[i]) continue;
38   solve(solve, i, 0, -1);
39 }
40 }

```

7.6. Eulerian Paths/Circuits

7.6.1. Undirected

Assuming we have an undirected *connected* graph, if all vertices have an *even* degree, an Eulerian circuit exists; otherwise, if we have two vertices of odd degree, an Eulerian path exists (o1 and o2 represent the two odd vertices).

Be careful about self-loops, they don't count towards the degree! In general this algorithm is pretty expensive, so perform it *just once* if possible.

```

1  int odds = 0, n, m, o1, o2;
2  V adj(n + 1, set<int>());
3  V p;
4
5  int main() {
6    // NOTE: do some pre-processing
7    // ...
8
9    if (odds == 1 || odds >= 3) {
10     cout << "IMPOSSIBLE" << "\n";
11     return 0;
12   }
13
14   auto solve = [&](auto self, int u) -> void {
15     auto &vs = adj[u];
16     while (!empty(vs)) {
17       int v = *begin(vs);
18       vs.erase(v);
19       adj[v].erase(u);
20       self(self, v);
21     }
22     p.push_back(u);
23   };
24   // WARN: assume one-indexed nodes
25   solve(solve, odds == 2 ? o1 : 1);
26
27   if (S(p) != m + 1) {
28     cout << "IMPOSSIBLE" << "\n";
29     return 0;
30   }
31   for (int i = 0; i < m + 1; i++) cout << p[i] << " \n"[i == m];
32 }

```

7.6.2. Directed

A directed graph has an Eulerian circuit if and only if it is *connected* and each vertex has the same in-degree as out-degree. A directed graph has an Eulerian path if and only if it is connected and each vertex except two has an in-degree as out-degree.

One of the two should have an out-degree one greater than in-degree (*start vertex*), while the other has an in-degree one greater than out-degree (*end vertex*). The core algorithm is the same as that for the undirected graph.

7.7. Binary Lifting

Before we can perform binary lifting, we have to first construct a table `tab`, where `tab[i][h]` represents the 2^h -th ancestor of `i`.

```

1  int n;
2
3  int main() {

```

```

4  // WARN: assumes zero-indexed nodes
5  V adj(n, V());
6
7  auto ceil_log2 = [](int n) -> int {
8    int ans = 0, cur = 1;
9    while (cur < n) cur *= 2, ans++;
10   return ans;
11 };
12
13 int tab_height = ceil_log2(n);
14 vector depth(n, 0);
15 vector tab(n, vector(tab_height + 1, -1));
16
17 auto build_tab = [&](auto self, int u, int pu) -> void {
18   tab[u][0] = pu;
19   for (auto &v : adj[u])
20     if (v != pu) {
21       depth[v] = depth[u] + 1;
22       self(self, v, u);
23     }
24 };
25 depth[0] = 1;
26 build_tab(build_tab, 0, -1);
27
28 for (int k = 1; k <= tab_height; k++) {
29   for (int u = 0; u < n; u++) {
30     if (tab[u][k - 1] != -1) {
31       tab[u][k] = tab[tab[u][k - 1]][k - 1];
32     }
33   }
34 }
35
36 auto get_parent = [&](int u, int k) -> int {
37   for (int h = 0; h <= tab_height; h++)
38     if ((k >> h) & 1) {
39       u = tab[u][h];
40       if (u == -1) break;
41     }
42   return u;
43 };
44
45 auto get_lca = [&](auto self, int u, int v) -> int {
46   if (depth[u] < depth[v]) return self(v, u);
47   u = get_parent(u, depth[u] - depth[v]);
48   if (u == v) return u;
49   for (int h = tab_height; h >= 0; h--) {
50     if (tab[u][h] != tab[v][h]) u = tab[u][h], v = tab[v][h];
51   }
52   return tab[u][0];
53 };
54
55 auto get_dist = [&](auto self, int u, int v) -> int {
56   if (depth[u] < depth[v]) return self(v, u);
57   int delta = depth[u] - depth[v], ans = delta;
58   u = get_parent(u, delta);
59   if (u == v) return delta;
60   for (int h = tab_height; h >= 0; h--) {
61     if (tab[u][h] != tab[v][h]) {
62       u = tab[u][h], v = tab[v][h];
63       ans += 2 * (1 << h);
64     }
65   }
66   return ans + 2;
67 };
68 }

```

7.8. Heavy Light Decomposition

If you want to update the value of node `u`, you need to update `pos[u]`. query needs to be tweaked depending on the problem.

```

1  int n;
2  constexpr int NMAX = 100'000;
3
4  int main() {
5    // WARN: assumes zero-indexed nodes
6    V adj(n, V());
7
8    V raw(n), s(n); // NOTE: `s` contains the value in each node

```

```

9    array<int, NMAX> par{}, depth{}, heavy{}, head{}, pos{};
10   fill(A(heavy), -1);
11   int cur_pos = 0;
12
13   auto dfs = [&](auto self, int u) -> int {
14     int subtree = 1, max_child = 0;
15     for (auto &v : adj[u])
16       if (par[u] != v) {
17         par[v] = u, depth[v] = depth[u] + 1;
18         int child = self(self, v);
19         if (child > max_child) max_child = child, heavy[u] = v;
20       }
21     return subtree;
22   };
23   dfs(dfs, 0);
24
25   auto dc = [&](auto self, int u, int h) -> void {
26     head[u] = h, pos[u] = cur_pos, raw[cur_pos] = s[u], cur_pos++;
27     if (heavy[u] != -1) self(self, heavy[u], h);
28     for (auto &v : adj[u])
29       if (par[u] != v && v != heavy[u]) {
30         self(self, v, v);
31       }
32   };
33   dc(dc, 0, 0);
34
35   SegTree tree(raw);
36   auto query = [&](int u, int v) -> int {
37     int ans = numeric_limits<int>::min();
38     for (; head[u] != head[v]; v = par[head[v]]) {
39       if (depth[head[u]] > depth[head[v]]) swap(u, v);
40       ans = tree.conquer(ans, tree.query(pos[head[v]], pos[v]));
41     }
42     if (depth[u] > depth[v]) swap(u, v);
43     return tree.conquer(ans, tree.query(pos[u], pos[v]));
44   };
45 }

```

7.9. Tree Hashing

You can hash a *rooted* tree like so. If it isn't rooted, you may find a centroid first before using it as a root to perform the hash. Some trees may have *two* centroids, in which case you have to retrieve two hashes. Among the two, put the smaller one before the larger one in a pair of integers (e.g., in `V<int>`).

```

1  auto hash = [](vector<vector<int>>& adj) -> array<Mint, 2> {
2    vector subs(n + 1, Mint());
3
4    // WARN: assumes one-indexed nodes
5    auto solve
6    = [&](auto self, int b = 31, int u = 1, int pu = 0, int d = 0) -> Mint {
7      deque<Mint> vs;
8      subs[u]++;
9
10     for (auto v : adj[u]) {
11       if (v == pu) continue;
12       vs.P(subs[v] * self(self, b, v, u, d + 1));
13       subs[u] += subs[v];
14     }
15     sort(A(vs), [&](Mint &x, Mint &y) { return x.v < y.v; });
16
17     vs.push_front(d), vs.push_back(d);
18
19     Mint ans = 0;
20
21     F(i, 0, S(vs)) { ans += vs[i] * Mint(b).pow(i); }
22
23     return ans;
24   };
25
26   return { solve(solve, 31), solve(solve, 39) };
27 };

```

7.10. Centroid Decomposition

`ancestors[u]` returns the list of ancestors of `u` (along with the distance) as we go up the centroid decomposition tree.

```
1 int main() {
2   // WARN: assumes one-indexed nodes
3   cin >> n;
4   V adj(n + 1, V());
5
6   F(_, 0, n - 1) {
7     int u, v;
8     cin >> u >> v;
9     adj[u].P(v);
10    adj[v].P(u);
11  }
12
13  V seen(n + 1), subt(n + 1, 0);
14  V ancestors(n + 1, V<ii>());
15
16  auto build_subt = [&](auto self, int u, int pu) -> int {
17    subt[u] = 1;
18    for (auto v : adj[u]) {
19      if (v == pu || seen[v]) continue;
20      subt[u] += self(self, v, u);
21    }
22    return subt[u];
23  };
24
25  // WARN: returns one centroid only (there might be two)
26  auto get_centroid = [&](auto self, int u, int pu, int tree_size) -> int {
27    for (auto v : adj[u]) {
28      if (v == pu || seen[v]) continue;
29      if (2 * subt[v] > tree_size) return self(self, v, u, tree_size);
30    }
31    return u;
32  };
33
34  auto build_ancestor
35    = [&](auto self, int u, int pu, int centroid, int cur_d = 1) -> void {
36    for (auto v : adj[u]) {
37      if (v == pu || seen[v]) continue;
38      self(self, v, u, centroid, cur_d + 1);
39    }
40    ancestors[u].P({ centroid, cur_d });
41  };
42
43  auto centroid_decomp = [&](auto self, int init = 1) -> void {
44    int centroid
45      = get_centroid(get_centroid, init, 0, build_subt(build_subt, init, 0));
46    for (auto v : adj[centroid]) {
47      if (seen[v]) continue;
48      build_ancestor(build_ancestor, v, centroid, centroid);
49    }
50    seen[centroid] = true;
51    for (auto v : adj[centroid]) {
52      if (seen[v]) continue;
53      self(self, v);
54    }
55  };
56 }
```

7.11. 2-SAT

```
1 // WARN: assumes zero-indexed nodes
2 class two_sat {
3   int n;
4   vector<vector<int>>> ori, rev;
5
6   public:
7   two_sat(int n)
8     : n(n) {
9     ori.assign(2 * n, {});
10    rev.assign(2 * n, {});
11  }
12  void add_clause(int u, bool u_pos, int v, bool v_pos) {
13    ori[2 * u + u_pos].push_back(2 * v + (!v_pos));
14    ori[2 * v + v_pos].push_back(2 * u + (!u_pos));
15    rev[2 * v + (!v_pos)].push_back(2 * u + u_pos);
16    rev[2 * u + (!u_pos)].push_back(2 * v + v_pos);
17  }
18  bool is_satisfiable() {
19    int color = 1;
20    stack<int> stk;
```

```
21    vector<int> seen(2 * n), colors(2 * n);
22    auto build_stk = [&](auto build_stk, int u) -> void {
23      seen[u] = true;
24      for (auto &v : ori[u])
25        if (!seen[v]) {
26          build_stk(build_stk, v);
27        }
28      stk.push(u);
29    };
30    for (int u = 0; u < 2 * n; u++)
31      if (!seen[u]) build_stk(build_stk, u);
32    auto build_scc = [&](auto build_scc, int u) -> void {
33      colors[u] = color, seen[u] = true;
34      for (auto &v : rev[u])
35        if (!colors[v]) {
36          build_scc(build_scc, v);
37        }
38    };
39    while (!stk.empty()) {
40      int u = stk.top();
41      stk.pop();
42      if (!colors[u]) {
43        build_scc(build_scc, u);
44        color++;
45      }
46    }
47    for (int i = 0; i < n; i++) {
48      if (colors[2 * i] == colors[2 * i + 1]) return false;
49    }
50    return true;
51  };
52 }
```

7.12. Small-To-Large Merging

Instead, of each node having its own set of objects, try to reuse the children sets as much as possible. This reduces the number of swaps/inserts and improves the constant factor. The following snippet solves CSES’s Distinct Colors, which asks you to find the number of distinct colors in each subtree.

```
1 void solve_t() {
2   cin >> n;
3
4   vector<set<int>> col(n + 1);
5   vector adj(n + 1, vector<int>());
6   vector ans(n + 1, 0);
7   vector c(n + 1, 0);
8
9   G(u, 1, n) { cin >> c[u]; }
10
11  F(_, 0, n - 1) {
12    int u, v;
13    cin >> u >> v;
14    adj[u].P(v);
15    adj[v].P(u);
16  }
17
18  // auto merge = [&](set<int> &s, set<int> &t) -> void {
19  //   if (S(s) < S(t)) swap(s, t);
20  //   for (auto &c : t) s.I(c);
21  //   t.clear();
22  // };
23
24  auto solve = [&](auto self, int u = 1, int pu = 0) -> set<int> * {
25    vector<set<int> *> ptrs;
26
27    for (auto v : adj[u]) {
28      if (v == pu) continue;
29      ptrs.P(self(self, v, u));
30    }
31
32    if (empty(ptrs)) {
33      col[u].I(c[u]), ans[u] = 1;
34      return &col[u];
35    }
36
37    int mxi = 0, mxv = S(*ptrs[0]);
```

```
39    F(i, 1, S(ptrs)) {
40      if (S(*ptrs[i]) > mxv) {
41        mxv = S(*ptrs[i]);
42        mxi = i;
43      }
44    }
45
46    F(i, 0, S(ptrs)) {
47      if (i == mxi) continue;
48      for (auto x : *ptrs[i]) {
49        ptrs[mxi]->insert(x);
50      }
51      ptrs[i]->clear();
52    }
53
54    ptrs[mxi]->insert(c[u]);
55    ans[u] = S(*ptrs[mxi]);
56
57    return ptrs[mxi];
58  };
59  solve(solve);
60
61  G(u, 1, n) { cout << ans[u] << N(u, n); }
62 }
```

8. Flows

8.1. Edmonds-Karp

Suppose we have already set up adj and cap, where cap represents the capacity of each edge; we may use this algorithm to determine the maximum flow in a flow network. When adding an edge, we need to include both (u, v) and (v, u) in adj, however, we’ll only set the capacity of (u, v) to whatever it is.

To determine the **minimum cut**, find all vertices that can be reached from the source, using only edges with positive residual capacity.

```
1 int main() {
2   V adj(n, V()), cap(n, V(n, 0));
3
4   auto bfs = [&](int s, int t, vector<int> &par) -> int {
5     par.assign(ssize(adj), -1), par[s] = -2;
6     queue<i> q;
7     q.push({ s, 0 });
8     while (!empty(q)) {
9       auto [u, flow] = q.front();
10      q.pop();
11      for (auto &v : adj[u]) {
12        if (par[v] != -1 || cap[u][v] == 0) continue;
13        par[v] = u;
14        int new_flow = min(flow, cap[u][v]);
15        if (v == t) return new_flow;
16        q.push({ v, new_flow });
17      }
18    }
19    return 0;
20  };
21
22  // WARN: you may need to alter `int` to `ll` depending on the max flow
23  auto max_flow = [&](int s, int t) -> int {
24    int flow = 0, new_flow = 0;
25    vector par(ssize(adj), 0);
26    while ((new_flow = bfs(s, t, par))) {
27      flow += new_flow;
28      int u = t;
29      while (u != s) {
30        int pu = par[u];
31        cap[pu][u] -= new_flow, cap[u][pu] += new_flow;
32        u = pu;
33      }
34    }
35    return flow;
36  };
37 }
```

8.2. Dinic’s Algorithm

Remember to reset all the edges if you need to compute the flow to some new destination. It’s also possible to get the incremental flow by adding edges and calling flow repeatedly.

```
1 struct FlowEdge {
2     int v, u;
3     ll cap, flow = 0;
4     FlowEdge(int v, int u, ll cap)
5         : v(v)
6         , u(u)
7         , cap(cap) {}
8 };
9
10 struct Dinic {
11     const ll flow_inf = OO;
12     vector<FlowEdge> edges;
13     vector<vector<int>>> adj;
14
15     int n, m = 0;
16     int s, t;
17
18     vector<int> level, ptr;
19     queue<int> q;
20
21     Dinic(int n, int s)
22         : n(n)
23         , s(s) {
24         adj.resize(n);
25         level.resize(n);
26         ptr.resize(n);
27     }
28
29     void add_edge(int v, int u, ll cap) { // NOTE: add edge (v -> u)
30         edges.emplace_back(v, u, cap);
31         edges.emplace_back(u, v, 0);
32         adj[v].push_back(m);
33         adj[u].push_back(m + 1);
34         m += 2;
35     }
36
37     bool bfs() {
38         while (!q.empty()) {
39             int v = q.front();
40             q.pop();
41             for (int id : adj[v]) {
42                 if (edges[id].cap == edges[id].flow) continue;
43                 if (level[edges[id].u] != -1) continue;
44                 level[edges[id].u] = level[v] + 1;
45                 q.push(edges[id].u);
46             }
47         }
48         return level[t] != -1;
49     }
50
51     ll dfs(int v, ll pushed) {
52         if (pushed == 0) return 0;
53         if (v == t) return pushed;
54         for (int &cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
55             int id = adj[v][cid];
56             int u = edges[id].u;
57             if (level[v] + 1 != level[u]) continue;
58             ll tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
59             if (tr == 0) continue;
60             edges[id].flow += tr;
61             edges[id ^ 1].flow -= tr;
62             return tr;
63         }
64         return 0;
65     }
66
67     ll flow(int t_) {
68         t = t_;
69         ll f = 0;
70         while (true) {
71             fill(A(level), -1);
72             level[s] = 0;
73             q.push(s);
```

```
74         if (!bfs()) break;
75         fill(A(ptr), 0);
76         while (ll pushed = dfs(s, flow_inf)) {
77             f += pushed;
78         }
79     }
80     return f;
81 }
82 ;
```

9. Sparse Table

```
1 template <typename T>
2 class SparseTable {
3     const int K = 25;
4     int n;
5     vector<vector<T>> st;
6     T conquer(T x, T y) { return min(x, y); } // WARN: rmb to tweak
7 public:
8     SparseTable(vector<T> &a)
9         : n(S(a))
10         , st(K + 1, vector<T>(n)) {
11         copy(all(a), begin(st[0]));
12         for (int i = 1; i <= K; i++) {
13             for (int j = 0; j + (1 << i) <= n; j++) {
14                 st[i][j] = conquer(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
15             }
16         }
17     }
18     T query(int l, int r) {
19         int i = __lg(r - l + 1);
20         return conquer(st[i][l], st[i][r - (1 << i) + 1]);
21     }
22 };
```

10. Fenwick Tree

10.1. One-Dimensional Queries

```
1 // NOTE: assumes one-indexed queries
2 template <typename T = int>
3 class FenwickTree {
4     int n;
5     vector<T> tree;
6
7 public:
8     FenwickTree(int tree_size)
9         : n(tree_size) {
10         tree.assign(n + 1, {});
11     }
12     void point_add(int i, T addend) {
13         for (; i <= n; i += lso(i)) tree[i] += addend;
14     }
15     void range_add(int l, int r, T addend) {
16         point_add(l, addend);
17         point_add(r + 1, -addend);
18     }
19     T point_ask(int i) {
20         T sum = {};
21         for (; i := lso(i)) sum += tree[i];
22         return sum;
23     }
24     T range_ask(int x, int y) { return point_ask(y) - point_ask(x - 1); }
25 };
```

10.2. Two-Dimensional Queries

```
1 // NOTE: assumes one-indexed queries
2 template <typename T = int>
3 class FenwickTree {
4     int height;
5     int width;
6     vector<vector<T>>> tree;
7
8 public:
```

```
9     FenwickTree(int h, int w)
10         : height(h)
11         , width(w) {
12         tree.assign(height + 1, vector<T>(width + 1));
13     }
14     T point_ask(int x, int y) {
15         T ans = {};
16         for (; x := lso(x)) {
17             for (int i = y; i := lso(i)) {
18                 ans += tree[x][i];
19             }
20         }
21         return ans;
22     }
23     T area_ask(int x1, int y1, int x2, int y2) {
24         T ans = point_ask(x2, y2) + point_ask(x1 - 1, y1 - 1);
25         ans -= point_ask(x2, y1 - 1) + point_ask(x1 - 1, y2);
26         return ans;
27     }
28     void point_add(int x, int y, T addend) {
29         for (; x <= height; x += lso(x)) {
30             for (int i = y; i <= width; i += lso(i)) {
31                 tree[x][i] += addend;
32             }
33         }
34     }
35     void area_add(int x1, int y1, int x2, int y2, T addend) {
36         point_add(x1, y1, addend);
37         point_add(x2 + 1, y2 + 1, addend);
38         point_add(x2 + 1, y1, -addend);
39         point_add(x1, y2 + 1, -addend);
40     }
41 };
```

10.3. Range Addition/Queries

This is a tiny bit faster compared to using segment trees.

```
1 // NOTE: assumes one-indexed queries
2 template <typename T = ll>
3 class FenwickTree {
4     int n;
5     vector<T> tree;
6
7 public:
8     FenwickTree(int tree_size)
9         : n(tree_size) {
10         tree.assign(n + 1, {});
11     }
12     void point_add(int i, T addend) {
13         for (; i <= n; i += lso(i)) tree[i] += addend;
14     }
15     T point_ask(int i) {
16         T sum = {};
17         for (; i := lso(i)) sum += tree[i];
18         return sum;
19     }
20 };
21 class DoubleFenwick {
22     int n;
23     FenwickTree<ll> b1, b2;
24     ll prefix_sum(int idx) { return b1.point_ask(idx) * idx - b2.point_ask(idx); }
25
26 public:
27     DoubleFenwick(int n)
28         : n(n)
29         , b1(n)
30         , b2(n) {}
31     void range_add(int l, int r, ll x) {
32         b1.point_add(l, x);
33         b1.point_add(r + 1, -x);
34         b2.point_add(l, x * (l - 1));
35         b2.point_add(r + 1, -x * r);
36     }
37     ll range_query(int l, int r) { return prefix_sum(r) - prefix_sum(l - 1); }
38 };
```

11. Segment Tree

11.1. RMQ (Iterative)

It can help to avoid TLE in certain situations.

```
1  template<typename T>
2  class SegTree {
3  int n;
4  vector<T> a, tree;
5  void build() {
6      for (int i = n - 1; i >= 1; --i) {
7          tree[i] = conquer(tree[i << 1], tree[(i << 1) ^ 1]);
8      }
9  }
10
11 public:
12 SegTree(vector<T> &a)
13     : a(a) {
14         n = ssize(a);
15         tree.assign(2 * n, {});
16         for (int i = 0; i < n; i++) tree[i + n] = a[i];
17         build();
18     }
19 T conquer(T x, T y) { // WARN: rmb to tweak
20     return min(x, y);
21 }
22 void update(int i, T val) {
23     for (tree[i += n] = val; i > 1; i >>= 1) {
24         tree[i >> 1] = conquer(tree[i], tree[i ^ 1]);
25     }
26 }
27 T query(int l, int r) {
28     T ans = numeric_limits<T>::max(); // WARN: depends on `conquer`
29     for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
30         if (l & 1) ans = conquer(ans, tree[l++]);
31         if (r & 1) ans = conquer(ans, tree[--r]);
32     }
33     return ans;
34 }
35 };
```

11.2. RMQ (Recursive, Monoidal)

See the Lazy Propagation section for more examples of monoids.

```
1  // Point update range query segtree over arbitrary state
2  template<typename T>
3  struct Tree {
4      ll n;
5      vector<T> v;
6      Tree(ll a_n): n(a_n), v(a_n << 2ll, T::identity()) {}
7      void _build(ll i, ll l, ll r, const vector<T> &a) {
8          if(l == r) {
9              v[i] = a[l];
10             return;
11         }
12         ll m = (l + r) >> 1ll;
13         _build(i << 1ll, l, m, a);
14         _build((i << 1ll) | 1ll, m + 1ll, r, a);
15         v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
16     }
17     void _build(ll i, ll l, ll r, T a_v) {
18         if(l == r) {
19             v[i] = a_v;
20             return;
21         }
22         ll m = (l + r) >> 1ll;
23         _build(i << 1ll, l, m, a_v);
24         _build((i << 1ll) | 1ll, m + 1ll, r, a_v);
25         v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
26     }
27     void _build(ll i, ll l, ll r, const function<T(ll)> &init) {
28         if(l == r) {
29             v[i] = init(l);
30             return;
31         }
32         ll m = (l + r) >> 1ll;
33         _build(i << 1ll, l, m, init);
```

```
34     _build((i << 1ll) | 1ll, m + 1ll, r, init);
35     v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
36 }
37 void build(const vector<T> &a) {
38     _build(1ll, 0ll, n - 1ll, a);
39 }
40 void build(T a_v) {
41     _build(1ll, 0ll, n - 1ll, a_v);
42 }
43 void build(const function<T(ll)> &init) {
44     _build(1ll, 0ll, n - 1ll, init);
45 }
46 T _qry(ll i, ll l, ll r, ll ql, ll qr) {
47     if(ql > qr) return T::identity();
48     if(ql > r || qr < l) return T::identity();
49     if(ql == l && qr == r) return v[i];
50     ll m = (l + r) >> 1ll;
51     return _qry(i << 1ll, l, m, ql, min(qr, m)) + _qry((i << 1ll) | 1ll, m + 1ll, r, max(ql, m + 1ll), qr);
52 }
53 T qry(ll ql, ll qr) {
54     return _qry(1ll, 0ll, n - 1ll, ql, qr);
55 }
56 void _upd(ll i, ll l, ll r, ll qi, const T &nv) {
57     if(l == r) {
58         v[i] = nv;
59         return;
60     }
61     ll m = (l + r) >> 1ll;
62     if(qi <= m) _upd(i << 1ll, l, m, qi, nv);
63     else _upd((i << 1ll) | 1ll, m + 1ll, r, qi, nv);
64     v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
65 }
66 void upd(ll qi, const T &nv) {
67     _upd(1ll, 0ll, n - 1ll, qi, nv);
68 }
69 };
70
71 /*
72 This is the Monoid, the state of the segment tree
73 Define as many attributes as you want in the constructor
74 Then, define how to combine to Mono objects. For example,
75 one could define it as addition, multiplication, string concatenation,
76 or combining substates in a DnC maximum subarray sum implementation.
77 ! Mono must be a Monoid. That is, it must satisfy:
78 ! (1) Associativity (e.g., (a+b)+c = a+(b+c) for addition)
79 ! (2) Existence of an Identity Element (e.g., 0 + x = x for addition), and
80 ! (3) Closure (e.g., a+b returns another integer if both a,b are integers)
81 */
82 struct Mono {
83     ll v;
84     Mono(ll a_v): v(a_v) {};
85     inline Mono operator+(const Mono &o) const {
86         return Mono(min(v, o.v));
87     }
88     static inline Mono identity() {
89         return Mono(INF(1ll));
90     }
91 };
```

11.3. Lazy Propagation (Default: Range Add+Min Query)

```
1  // Range update range query segtree over arbitrary state
2  template<typename T, typename U>
3  struct Tree {
4      ll n;
5      vector<T> v;
6      vector<U> lazy;
7      Tree(ll a_n): n(a_n), v(a_n << 2ll, T::identity()), lazy(a_n << 2ll, U::identity()) {}
8      void _build(ll i, ll l, ll r, const vector<T> &a) {
9          if(l == r) {
10             v[i] = a[l];
11             lazy[i] = U::identity();
12             return;
13         }
14         ll m = (l + r) >> 1ll;
15         _build(i << 1ll, l, m, a);
16         _build((i << 1ll) | 1ll, m + 1ll, r, a);
17         v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
18         lazy[i] = U::identity();
```

```
19     }
20     void _build(ll i, ll l, ll r, const T &a_v) {
21         if(l == r) {
22             v[i] = a_v;
23             lazy[i] = U::identity();
24             return;
25         }
26         ll m = (l + r) >> 1ll;
27         _build(i << 1ll, l, m, a_v);
28         _build((i << 1ll) | 1ll, m + 1ll, r, a_v);
29         v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
30         lazy[i] = U::identity();
31     }
32     void _build(ll i, ll l, ll r, const function<T(ll)> &init) {
33         if(l == r) {
34             v[i] = init(l);
35             lazy[i] = U::identity();
36             return;
37         }
38         ll m = (l + r) >> 1ll;
39         _build(i << 1ll, l, m, init);
40         _build((i << 1ll) | 1ll, m + 1ll, r, init);
41         v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
42         lazy[i] = U::identity();
43     }
44     void build(const vector<T> &a) {
45         _build(1ll, 0ll, n - 1ll, a);
46     }
47     void build(const T &a_v) {
48         _build(1ll, 0ll, n - 1ll, a_v);
49     }
50     void build(const function<T(ll)> &init) {
51         _build(1ll, 0ll, n - 1ll, init);
52     }
53     void push(ll i, ll l, ll r) {
54         if(lazy[i].is_identity()) {
55             return;
56         }
57         if(l == r) {
58             lazy[i] = U::identity();
59             return;
60         }
61         ll m = (l + r) >> 1ll;
62         lazy[(i << 1ll)] = lazy[(i << 1ll)] + lazy[i];
63         v[(i << 1ll)] = lazy[i].upd(l, m, v[(i << 1ll)]);
64         lazy[(i << 1ll) | 1ll] = lazy[(i << 1ll) | 1ll] + lazy[i];
65         v[(i << 1ll) | 1ll] = lazy[i].upd(m + 1ll, r, v[(i << 1ll) | 1ll]);
66         v[i] = v[(i << 1ll)] + v[(i << 1ll) | 1ll];
67         lazy[i] = U::identity();
68     }
69     T _qry(ll i, ll l, ll r, ll ql, ll qr) {
70         if(ql > r || qr < l) return T::identity();
71         push(i, l, r);
72         if(ql == l && qr == r) return v[i];
73         ll m = (l + r) >> 1ll;
74         T res = _qry(i << 1ll, l, m, ql, min(qr, m)) + _qry((i << 1ll) | 1ll, m + 1ll, r, max(ql, m + 1ll), qr);
75         v[i] = v[(i << 1ll)] + v[(i << 1ll) | 1ll];
76         return res;
77     }
78     T qry(ll ql, ll qr) {
79         return _qry(1ll, 0ll, n - 1ll, ql, qr);
80     }
81     void _upd(ll i, ll l, ll r, ll ql, ll qr, const U &updfn) {
82         if(ql > r || qr < l) return;
83         push(i, l, r);
84         if(ql == l && qr == r) {
85             lazy[i] = lazy[i] + updfn;
86             v[i] = updfn.upd(l, r, v[i]);
87             return;
88         }
89         ll m = (l + r) >> 1ll;
90         _upd(i << 1ll, l, m, ql, min(m, qr), updfn);
91         _upd((i << 1ll) | 1ll, m + 1ll, r, max(ql, m + 1ll), qr, updfn);
92         v[i] = v[i << 1ll] + v[(i << 1ll) | 1ll];
93     }
94     void upd(ll ql, ll qr, const U &updfn) {
95         _upd(1ll, 0ll, n - 1ll, ql, qr, updfn);
96     }
97 };
98
```

```
99
100 /*
101 This is the Monoid, the state of the segment tree
102 Define as many attributes as you want in the constructor
103 Then, define how to combine to Mono objects. For example,
104 one could define it as addition, multiplication, string concatenation,
105 or combining substates in a DnC maximum subarray sum implementation.
106 ! Mono must be a Monoid. That is, it must satisfy:
107 ! (1) Associativity (e.g., (a+b)+c = a+(b+c) for addition)
108 ! (2) Existence of an Identity Element (e.g., 0 + x = x for addition), and
109 ! (3) Closure (e.g., a+b returns another integer if both a,b are integers)
110 */
111 struct Mono {
112     ll v;
113     Mono(ll a_v): v(a_v) {};
114     inline Mono operator+(const Mono& o) const {
115         return Mono(v + o.v);
116     }
117     static inline Mono identity() {
118         return Mono(0ll);
119     }
120 };
121
122 /*
123 This is the Lazy Segtree Update Strategy
124 Define as many attributes as you want in the constructor
125
126 Then, upd implements a range update over a single node over [l, r]
127 Given an old state (of type const Mono&) and the interval [l, r], what would
128 the new state (of type Mono) be?
129 */
130 struct Upd {
131     ll k;
132     Upd(ll a_k): k(a_k) {};
133     Mono upd(ll l, ll r, const Mono& old) const {
134         return Mono(old.v + k * (r - l + 1ll));
135     }
136     Upd operator+(const Upd& o) const {
137         return Upd(k + o.k);
138     }
139     bool is_identity() const {
140         return k == 0ll;
141     }
142     static inline Upd identity() {
143         return Upd(0ll);
144     }
145 };
```

11.3.1. Range Addition/Assignment (Min Query)

```
1 // Range min (tested!)
2 struct Mono {
3     ll v;
4
5     Mono(ll a_v): v(a_v) {};
6
7     inline Mono operator+(const Mono& o) const {
8         return Mono(min(v, o.v));
9     }
10
11     static inline Mono identity() {
12         return Mono(INF(ll));
13     }
14 };
15
16 // Increment/assign (tested!)
17 struct Upd {
18     ll k, v;
19     bool assgn;
20     Upd(ll a_k, ll a_v, bool a_assgn): k(a_k), v(a_v), assgn(a_assgn) {};
21
22     Mono upd(ll l, ll r, const Mono& old) const {
23         Mono nv = Mono(old.v);
24
25         nv.v += k;
26         if(assgn) {
27             nv.v = v;
28         }
29 }
```

```
30     return nv;
31 }
32
33 Upd operator+(const Upd& o) const {
34     ll nk = 0ll, nv = 0ll;
35     bool nassgn = false;
36     if(o.assgn) {
37         nassgn = true;
38         nv = o.v;
39     } else if(assgn) {
40         nassgn = true;
41         nv = v + o.k;
42     } else {
43         nk = k + o.k;
44     }
45
46     return Upd(nk, nv, nassgn);
47 }
48
49 bool is_identity() const {
50     return k == 0ll && !assgn;
51 }
52
53 static inline Upd identity() {
54     return Upd(0ll, 0ll, false);
55 }
56 };
```

11.3.2. Range Addition/Assignment (Sum Query)

```
1 // Range sum (tested!)
2 struct Mono {
3     ll v;
4
5     Mono(ll a_v): v(a_v) {};
6
7     inline Mono operator+(const Mono& o) const {
8         return Mono(v + o.v);
9     }
10
11     static inline Mono identity() {
12         return Mono(0ll);
13     }
14 };
15
16 // Increment/assign (tested!)
17 struct Upd {
18     ll k, v;
19     bool assgn;
20     Upd(ll a_k, ll a_v, bool a_assgn): k(a_k), v(a_v), assgn(a_assgn) {};
21
22     Mono upd(ll l, ll r, const Mono& old) const {
23         Mono nv = Mono(old.v);
24
25         nv.v += k * (r - l + 1ll);
26         if(assgn) {
27             nv.v = v * (r - l + 1ll);
28         }
29
30         return nv;
31 }
32
33 Upd operator+(const Upd& o) const {
34     ll nk = 0ll, nv = 0ll;
35     bool nassgn = false;
36     if(o.assgn) {
37         nassgn = true;
38         nv = o.v;
39     } else if(assgn) {
40         nassgn = true;
41         nv = v + o.k;
42     } else {
43         nk = k + o.k;
44     }
45
46     return Upd(nk, nv, nassgn);
47 }
48
49 bool is_identity() const {
```

```
50     return k == 0ll && !assgn;
51 }
52
53 static inline Upd identity() {
54     return Upd(0ll, 0ll, false);
55 }
56 };
```

11.3.3. Range Addition/Multiplication (Sum Query)

```
1 ll md; // ! UPDATE THIS FIELD DURING RUNTIME
2 // Range sum (modular)
3 struct Mono {
4     ll v;
5
6     Mono(ll a_v): v(a_v) {};
7
8     inline Mono operator+(const Mono& o) const {
9         ll nv = v;
10        nv += o.v;
11        if(nv >= md) nv -= md;
12        return Mono(nv);
13    }
14
15    static inline Mono identity() {
16        return Mono(0ll);
17    }
18 };
19
20 // Range add/multiply (modular) (tested!)
21 // ! be careful: this assumes that the multiplier is nonnegative
22 struct Upd {
23     ll add, mult;
24     Upd(ll a_add, ll a_mult): add((a_add % md + md) % md), mult(a_mult % md) {};
25
26     Mono upd(ll l, ll r, const Mono& old) const {
27         Mono nv = Mono(old.v);
28
29         nv.v *= mult;
30         nv.v %= md;
31
32         nv.v += (add * (r - l + 1ll)) % md;
33         if(nv.v >= md) nv.v -= md;
34
35         return nv;
36     }
37 }
38
39 Upd operator+(const Upd& o) const {
40     /*
41     x -> mx + a -> m2(mx + a) + a2
42     = (m2 * m)(x) + (m2 a + a2)
43     */
44
45     return Upd((o.mult * add + o.add), (o.mult * mult));
46 }
47
48 bool is_identity() const {
49     return add == 0ll && mult == 1ll % md;
50 }
51
52 static inline Upd identity() {
53     return Upd(0ll, 1ll);
54 }
55 };
```

11.3.4. Polynomial Queries

Add [base, base+1, base+2, ...] to the range [l, r].

```
1 // Range sum
2 struct Mono {
3     ll v;
4
5     Mono(ll a_v): v(a_v) {};
6
7     inline Mono operator+(const Mono& o) const {
8         return Mono(v + o.v);
```

```

9   }
10
11   static inline Mono identity() {
12       return Mono(0ll);
13   }
14 };
15
16 // Increment by linear function
17 struct Upd {
18     ll b, c;
19     Upd(ll a_b, ll a_c): b(a_b), c(a_c) {};
20
21     Mono upd(ll l, ll r, const Mono& old) const {
22         return Mono(old.v + c * (r - l + 1ll) + b * (((l + r) * (r - l + 1ll)) >> 1ll)); // ! CAREFUL: your
23         application of a lazy update was wrong --- forgot to multiply by (r - l + 1)
24     }
25
26     Upd operator+(const Upd& o) const {
27         return Upd(b + o.b, c + o.c);
28     }
29
30     bool is_identity() const {
31         return b == 0ll && c == 0ll;
32     }
33
34     static inline Upd identity() {
35         return Upd(0ll, 0ll);
36     };

```

11.3.5. Range Addition/Assignment (Count Minimums)

query returns the minimum value in a range, as well as the corresponding count.

```

1 // Range min with frequency (tested)
2 struct Mono {
3     ll v, cnt;
4
5     Mono(ll a_v, ll a_cnt): v(a_v), cnt(a_cnt) {};
6
7     inline Mono operator+(const Mono& o) const {
8         ll nv = INF(ll), ncnt = 0ll;
9
10        if(v > o.v) {
11            nv = o.v;
12            ncnt = o.cnt;
13        } else if(v < o.v) {
14            nv = v;
15            ncnt = cnt;
16        } else {
17            nv = v;
18            ncnt = cnt + o.cnt;
19        }
20
21        return Mono(nv, ncnt);
22    }
23
24    static inline Mono identity() {
25        return Mono(INF(ll), 0ll);
26    }
27 };
28
29 // Increment/assign (tested!)
30 struct Upd {
31     ll k, v;
32     bool assgn;
33     Upd(ll a_k, ll a_v, bool a_assgn): k(a_k), v(a_v), assgn(a_assgn) {};
34
35     Mono upd(ll l, ll r, const Mono& old) const {
36         Mono nv = Mono(old.v, old.cnt);
37
38         nv.v += k;
39         if(assgn) {
40             nv.v = v;
41             nv.cnt = r - l + 1ll;
42         }
43
44         return nv;
45     };

```

```

46
47 Upd operator+(const Upd& o) const {
48     ll nk = 0ll, nv = 0ll;
49     bool nassgn = false;
50     if(o.assgn) {
51         nassgn = true;
52         nv = o.v;
53     } else if(assgn) {
54         nassgn = true;
55         nv = v + o.k;
56     } else {
57         nk = k + o.k;
58     }
59
60     return Upd(nk, nv, nassgn);
61 }
62
63 bool is_identity() const {
64     return k == 0ll && !assgn;
65 }
66
67 static inline Upd identity() {
68     return Upd(0ll, 0ll, false);
69 }
70 };

```

11.4. Persistent Segment Tree

Stores earlier versions of the tree. To create a new version, use cpy_root.

```

1 template <typename T>
2 class PersistentTree {
3     struct Node {
4         Node *l, *r;
5         T s;
6         Node(T s)
7             : l(nullptr)
8             , r(nullptr)
9             , s(s) {}
10        Node(Node *u)
11            : l(u->l)
12            , r(u->r)
13            , s(u->s) {}
14        Node(Node *l, Node *r)
15            : l(l)
16            , r(r)
17            , s() { // WARN: depends on `conquer`
18            if (!l && r) s = r->s;
19            if (l && !r) s = l->s;
20            if (l && r) s = conquer(l->s, r->s);
21        }
22    };
23    int n = 0;
24    vector<Node*> roots;
25    Node *build(const vector<T> &a, int tl, int tr) {
26        if (tl == tr) return new Node(a[tl]);
27        int tm = (tl + tr) / 2;
28        return new Node(build(a, tl, tm), build(a, tm + 1, tr));
29    }
30    T query(Node *u, int tl, int tr, int l, int r) {
31        if (l > r) return {}; // WARN: depends on `conquer`
32        if (l == tl && r == tr) return u->s;
33        int tm = (tl + tr) / 2;
34        T ls = query(u->l, tl, tm, l, min(r, tm));
35        T rs = query(u->r, tm + 1, tr, max(l, tm + 1), r);
36        return conquer(ls, rs);
37    }
38    Node *update(Node *u, int tl, int tr, int pos, T s) {
39        if (tl == tr) return new Node(s);
40        int tm = (tl + tr) / 2;
41        if (pos <= tm) return new Node(update(u->l, tl, tm, pos, s), u->r);
42        else return new Node(u->l, update(u->r, tm + 1, tr, pos, s));
43    }
44
45 public:
46     PersistentTree(vector<T> &a, int root_cnt, int init)
47         : roots(root_cnt) {
48         n = ssize(a);
49         roots[init] = build(a, 0, n - 1);

```

```

50     }
51     static T conquer(T x, T y) { // WARN: rmb to tweak
52         return x + y;
53     }
54     void update(int root, int pos, T s) {
55         roots[root] = update(roots[root], 0, n - 1, pos, s);
56     }
57     void cpy_root(int src, int dst) { roots[dst] = new Node(roots[src]); }
58     T query(int root, int l, int r) { return query(roots[root], 0, n - 1, l, r); }
59 };
60
61 int n, q, qtype, root_cnt = 0, QMAX = 100'000;
62
63 int main() {
64     cin.tie(nullptr)->sync_with_stdio(false);
65
66     cin >> n;
67     V a(n + 1, 0LL);
68
69     G(i, 1, n) cin >> a[i];
70     PersistentTree ptree(a, QMAX + 1, root_cnt);
71
72     cin >> q;
73
74     while (q--) {
75         cin >> qtype;
76         if (qtype == 1) {
77             int idx, v;
78             cin >> idx >> v, root_cnt++;
79             ptree.cpy_root(root_cnt - 1, root_cnt);
80             ptree.update(root_cnt, idx, v);
81         } else {
82             int hist_num, l, r;
83             cin >> hist_num >> l >> r;
84             cout << ptree.query(hist_num, l, r) << '\n';
85         }
86     }
87     return 0;
88 }

```

11.5. Merge Sort Tree

Stores the result of merge sort operations; may be used to answer queries such as finding the number of items $\leq x$ within a range.

```

1 struct Data {
2     vector<int> raw;
3 };
4 template <typename T = Data>
5 class MergeSortTree {
6     int n;
7     vector<T> a, tree;
8     void build(int i, int l, int r) {
9         if (l == r) tree[i] = a[l];
10        else {
11            int m = (l + r) / 2;
12            build(2 * i, l, m);
13            build(2 * i + 1, m + 1, r);
14            tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
15        }
16    }
17    void update(int i, int tl, int tr, int pos, T val) {
18        if (tl == tr) tree[i] = val;
19        else {
20            int tm = (tl + tr) / 2;
21            if (pos <= tm) update(2 * i, tl, tm, pos, val);
22            else update(2 * i + 1, tm + 1, tr, pos, val);
23            tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
24        }
25    }
26    int query(int i, int tl, int tr, int l, int r, int x) {
27        if (l > r) return {}; // WARN: depends on `conquer`
28        if (l == tl && r == tr) {
29            int pos = upper_bound(A(tree[i].raw), x) - begin(tree[i].raw);
30            return pos;
31        }
32        int tm = (tl + tr) / 2;
33        int lans = query(2 * i, tl, tm, l, min(r, tm), x);
34        int rans = query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, x);

```

```

35     return lans + rans;
36 }
37
38 public:
39 MergeSortTree(vector<T> &a)
40 : a(a) {
41     n = ssize(a);
42     tree.assign(4 * n, {});
43     build(1, 0, n - 1);
44 }
45 T conquer(Data x, Data y) { // WARN: rmb to tweak
46     Data ans;
47     merge(A(x.raw), A(y.raw), back_inserter(ans.raw));
48     return ans;
49 }
50 void update(int pos, T val) { update(1, 0, n - 1, pos, val); }
51 int query(int l, int r, int x) { return query(1, 0, n - 1, l, r, x); }
52 };

```

11.6. Li Chao Tree

Stores a collection of lines. Given an x coordinate, returns the minimum y coordinate on any of the lines. In this sense, it maintains the convex hull of the lines.

```

1  using namespace std;
2  typedef long long ll;
3  typedef vector<ll> vll;
4  typedef vector<vll> vvll;
5  typedef pair<ll, ll> pll;
6  #define INF(dtype) numeric_limits<dtype>::max()
7
8  // Create a struct for linear functions
9  // First argument: slope, second argument: y-intercept
10 struct Line {
11     ll m, b;
12
13     ll evaluate(ll x) const {
14         return m * x + b;
15     }
16 };
17
18 /*
19 Maintain the minimum line in the li chao tree
20 */
21 class Tree {
22 public:
23     Tree *lt, *rt;
24     ll l, r;
25     Line v;
26
27     Tree(ll a_l, ll a_r): lt(nullptr), rt(nullptr), l(a_l), r(a_r), v({0ll, INF(ll)}) {}
28
29 void build() {
30     if(l == r) {
31         return;
32     }
33
34     ll m = (l + r) >> 1ll;
35
36     lt = new Tree(l, m);
37     rt = new Tree(m + 1ll, r);
38
39     lt->build();
40     rt->build();
41 }
42
43 void insert_line(Line nl) {
44     ll m = (l + r) >> 1ll;
45     bool less_l = nl.evaluate(l) < v.evaluate(l);
46     bool less_m = nl.evaluate(m + 1ll) < v.evaluate(m + 1ll);
47
48     /*
49     ! TODO why does swap work
50     ! and v = nl does not???
51
52     well, if we do v = nl,
53     we simply erase v from existence

```

```

54     we don't want that---we want to consider both
55
56     so nl is considered now
57     therefore we instead propagate v downward
58     */
59     if(less_m) swap(v, nl);
60
61     if(l == r) {
62         return;
63     } else if(less_l != less_m) {
64         lt->insert_line(nl);
65     } else {
66         rt->insert_line(nl);
67     }
68 }
69
70 ll evaluate(ll x) {
71     if(l == r) {
72         return v.evaluate(x);
73     }
74     ll m = (l + r) >> 1ll;
75
76     if(x <= m) {
77         return min(lt->evaluate(x), v.evaluate(x));
78     } else {
79         return min(rt->evaluate(x), v.evaluate(x));
80     }
81 }
82 };

```

12. Convolution

12.1. Number Theoretic Transform

```

1  constexpr ll MOD = 998'244'353, root = 62;
2  ll modpow(ll b, ll p) {
3      if(p == 0) return 1LL;
4      ll ans = modpow(b, p / 2);
5      ans = ((__int128)(ans)*ans) % MOD;
6      if(p & 1) ans = (ans * b) % MOD;
7      return ans;
8  }
9
10 void ntt(vector<ll> &a) {
11     int n = ssize(a), L = 31 - __builtin_clz(n);
12     static vector<ll> rt(2, 1);
13     for (static int k = 2, s = 2; k < n; k *= 2, s++) {
14         rt.resize(n);
15         ll z[] = { 1, modpow(root, MOD >> s) };
16         for (int i = k; i < 2 * k; i++) rt[i / 2] = rt[i / 2] * z[i & 1] % MOD;
17     }
18     vector<int> rev(n);
19     for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
20     for (int i = 0; i < n; i++)
21         if (i < rev[i]) swap(a[i], a[rev[i]]);
22     for (int k = 1; k < n; k *= 2)
23         for (int i = 0; i < n; i += 2 * k)
24             for (int j = 0; j < k; j++) {
25                 ll z = rt[j + k] * a[i + j + k] % MOD, &ai = a[i + j];
26                 a[i + j + k] = ai - z + (z > ai ? MOD : 0);
27                 ai += (ai + z >= MOD ? z - MOD : z);
28             }
29 }
30
31 vector<ll> conv(const vector<ll> &a, const vector<ll> &b) {
32     if (a.empty() || b.empty()) return {};
33     int s = ssize(a) + ssize(b) - 1, B = 32 - __builtin_clz(s), n = 1 << B;
34     int inv = modpow(n, MOD - 2);
35     vector<ll> left(a), right(b), out(n);
36     left.resize(n), right.resize(n);
37     ntt(left), ntt(right);
38     for (int i = 0; i < n; i++)
39         out[(i & (n - 1))] = (ll)left[i] * right[i] % MOD * inv % MOD;
40     ntt(out);
41     return { out.begin(), out.begin() + s };
42 }

```

13. Math

13.1. Extended Euclidean Algorithm

Finds x and y such that $ax + yb = \gcd(a, b)$.

```

1  class GCD {
2  static pair<ll, ll> solve(ll a, ll b) {
3      if (b == 0) return { 1, 0 };
4      auto [y, x] = solve(b, a % b);
5      y -= a / b * x;
6      return { x, y };
7  }
8
9  public:
10 static pair<ll, ll> extended_gcd(ll a, ll b) {
11     auto [x, y] = solve(a, b);
12     if (x * a + b * y == -gcd(a, b)) x *= -1, y *= -1;
13     return { x, y };
14 }
15 };

```

13.2. Mint

```

1  class Mint {
2  constexpr ll mod(ll x) const {
3      if (x < 0) x += MOD;
4      return x >= MOD ? x % MOD : x;
5  }
6
7  public:
8  static constexpr ll MOD = 1'000'000'007; // WARN: rmb to tweak
9  ll v;
10 constexpr ll pow(ll p) const {
11     if (!p) return 1;
12     ll ans = pow(p / 2);
13     return mod(p & 1 ? v * mod(ans * ans) : mod(ans * ans));
14 }
15 constexpr ll i() const { return pow(MOD - 2); }
16 constexpr Mint(ll v = 0)
17 : v(mod(v)) {}
18 constexpr Mint(const Mint &x)
19 : v(x.v) {}
20 constexpr Mint &operator++() {
21     v = mod(v + 1);
22     return *this;
23 }
24 constexpr Mint operator++(int) {
25     Mint _ = *this;
26     ++(*this);
27     return _;
28 }
29 constexpr Mint &operator--() {
30     v = mod(v - 1);
31     return *this;
32 }
33 constexpr Mint operator--(int) {
34     Mint _ = *this;
35     --(*this);
36     return _;
37 }
38 constexpr Mint &operator=(const Mint &y) {
39     v = y.v;
40     return *this;
41 }
42 constexpr Mint &operator+=(const Mint &y) {
43     v = mod(v + y.v);
44     return *this;
45 }
46 constexpr Mint &operator-=(const Mint &y) {
47     v = mod(v - y.v);
48     return *this;
49 }
50 constexpr Mint &operator*=(const Mint &y) {
51     v = mod(v * y.v);
52     return *this;
53 }
54 constexpr Mint &operator/=(const Mint &y) {
55     v = mod(v * y.i());

```

```

56     return *this;
57 }
58 friend constexpr Mint operator+(Mint x, const Mint &y) {
59     x += y;
60     return x;
61 }
62 friend constexpr Mint operator-(Mint x, const Mint &y) {
63     x -= y;
64     return x;
65 }
66 friend constexpr Mint operator*(Mint x, const Mint &y) {
67     x *= y;
68     return x;
69 }
70 friend constexpr Mint operator/(Mint x, const Mint &y) {
71     x /= y;
72     return x;
73 }
74 friend constexpr bool operator==(const Mint &x, const Mint &y) {
75     return x.v == y.v;
76 }
77 friend constexpr bool operator!=(const Mint &x, const Mint &y) {
78     return x.v != y.v;
79 }
80 friend ostream &operator<<(ostream &os, const Mint &x) { return os << x.v; }
81 ;

```

13.3. Sieve of Eratosthenes

```

1  constexpr int SIEVE_SIZE = 1'000'000; // WARN: rmb to tweak
2  bitset<SIEVE_SIZE + 1> bs;
3  vector<ll> primes;
4  void sieve() {
5      bs.set();
6      bs[0] = bs[1] = false;
7      for (ll i = 2; i <= SIEVE_SIZE; i++)
8          if (bs[i]) {
9              for (ll j = i * i; j <= SIEVE_SIZE; j += i) bs[j] = false;
10             primes.push_back(i);
11         }
12 }

```

13.3.1. Euler's Totient Function

$\varphi(n)$ computes the positive integers up to n that are coprime to n . We can compute it from the prime factors of n .

```

1  ll eulerPhi(ll n) {
2      ll ans = n;
3      for (int i = 0; i < S(primes) && primes[i] * primes[i] <= n; i++) {
4          if (n % primes[i] == 0) ans -= ans / primes[i];
5          while (n % primes[i] == 0) n /= primes[i];
6      }
7      return ans != 1 ? ans - ans / n : ans;
8  }

```

We can also compute it using a modified sieve.

```

1  const int SIEVE_SIZE = 1'000'000; // WARN: rmb to tweak
2  int eulerPhi[SIEVE_SIZE + 1];
3  void sieve() {
4      for (int i = 1; i <= SIEVE_SIZE; i++) eulerPhi[i] = i;
5      for (int i = 2; i <= SIEVE_SIZE; i++)
6          if (eulerPhi[i] == i)
7              for (int j = i; j <= SIEVE_SIZE; j += i) eulerPhi[j] -= eulerPhi[j] / i;
8  }

```

13.4. Fast Factorization

Runs in about $\mathcal{O}(n^{\frac{1}{3}})$ time, to be used sparingly.

```

1  ll modmul(ll x, ll y, ll mod) { return ((__int128)x * y) % mod; }
2
3  ll modpow(ll b, ll p, ll mod) {
4      if (!p) return 1;

```

```

5      ll ans = modpow(b, p / 2, mod) % mod;
6      ans = modmul(ans, ans, mod);
7      if (p & 1) ans = modmul(ans, b, mod);
8      return ans;
9  }
10
11 bool is_prime(ll n) {
12     if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
13     ll A[] = { 2, 325, 9375, 28178, 450775, 9780504, 1795265022 },
14             s = __builtin_ctzll(n - 1), d = n >> s;
15     for (ll a : A) {
16         ll p = modpow(a % n, d, n), i = s;
17         while (p != 1 && p != n - 1 && a % n && i-->) p = modmul(p, p, n);
18         if (p != n - 1 && i != s) return 0;
19     }
20     return 1;
21 }
22
23 ll rho(ll n) {
24     ll x = 0, y = 0, t = 30, prd = 2, i = 1, q;
25     auto f = [&](ll x) { return modmul(x, x, n) + i; };
26     while (t++ % 40 || __gcd(prd, n) == 1) {
27         if (x == y) x = ++i, y = f(x);
28         if ((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
29         x = f(x), y = f(f(y));
30     }
31     return __gcd(prd, n);
32 }
33
34 vector<ll> factor(ll n) {
35     if (n == 1) return { 1 };
36     auto solve = [&](auto self, ll n) -> V<ll> {
37         if (n == 1) return {};
38         if (is_prime(n)) return { n };
39         auto d = rho(n);
40         V<ll> a = self(self, d), b = self(self, n / d), ans;
41         for (auto x : a) ans.P(x);
42         for (auto y : b) ans.P(y);
43         return ans;
44     };
45     auto ans = solve(solve, n);
46     return ans;
47 }

```

13.5. Frobenius Number

The *Frobenius number* is the largest integer that *can't* be expressed as a non-negative linear combination of two coprime integers p and q . It is computed as $pq - p - q$.

13.6. Matrix

May be helpful for matrix multiplication problems.

```

1  constexpr ll MOD = 1'000'000'007;
2
3  template <typename T = ll>
4  struct Mat2D {
5      int n, m;
6      vector<vector<T>>> g;
7      Mat2D(int n, int m, bool identity = false)
8          : n(n)
9            , m(m)
10            , g(n, vector<T>(m)) {
11          if (n != m || !identity) return;
12          for (int i = 0; i < n; i++) {
13              g[i][i] = 1;
14          }
15      }
16      constexpr Mat2D pow(ll p) const {
17          if (!p) return Mat2D(this->n, this->m, true);
18          Mat2D ans = pow(p / 2);
19          return p & 1 ? (*this) * ans * ans : ans;
20      }
21      constexpr Mat2D &operator*=(const Mat2D &b) {
22          assert(this->m == b.n);
23          Mat2D c = Mat2D(this->n, b.m);
24          for (int i = 0; i < this->n; i++) {

```

```

25         for (int j = 0; j < b.m; j++) {
26             for (int k = 0; k < this->m; k++) {
27                 c.g[i][j] += this->g[i][k] * b.g[k][j];
28                 c.g[i][j] %= MOD; // WARN: may not be necessary
29             }
30         }
31     }
32     n = c.n, m = c.m, g = c.g;
33     return *this;
34 }
35 friend constexpr Mat2D operator*(Mat2D x, const Mat2D &y) {
36     x *= y;
37     return x;
38 }
39 };

```

13.7. Linear Algebra

Useful for dealing with systems of equations. Operations are assumed to be done in modulo p . Formally, operands are elements of $M_{i \times j}(\mathbb{Z}/p\mathbb{Z})$.

```

1  typedef long long ll;
2  typedef vector<ll> vll;
3  typedef vector<vll> vvll;
4
5  // Modular Arithmetic Tools needed for operations in Z/pZ
6  ll fpw(ll base, ll pw, ll md) {
7      base %= md;
8      if (pw == 0ll) return 1ll;
9      ll bsqr = fpw((base * base) % md, pw >> 1ll, md);
10
11     return (pw & 0b1ll ? (bsqr * base) % md : bsqr);
12 }
13
14 ll minv(ll n, ll md) {
15     return fpw(n, md - 2ll, md);
16 }
17
18 ll frac(ll num, ll den, ll md) {
19     return (num * minv(den, md)) % md;
20 }
21
22 // Prints a matrix
23 void matprint(const vvll& m) {
24     ll r = m.size();
25
26     if (r == 0ll) {
27         cerr << "empty list!" << endl;
28         return;
29     }
30
31     ll c = m[0ll].size();
32
33     for (ll i = 0ll; i < r; i++) {
34         for (ll j = 0ll; j < c; j++) cerr << m[i][j] << " ";
35         cerr << "\n";
36     }
37
38     cerr << flush;
39 }
40
41 // Multiplies two matrices M1 and M2
42 // order of operands follows linear algebra conventions :)
43 vvll matmul(const vvll& m1, const vvll& m2) {
44     ll nr = m1.size(), nc = m2[0].size();
45     ll maxk = m1.size(); // should be equal to m1[0].size()
46
47     vvll res(nr, vll(nc, 0ll));
48
49     for (ll r = 0; r < nr; r++) {
50         for (ll k = 0; k < maxk; k++) {
51             for (ll c = 0; c < nc; c++) {
52                 res[r][c] += m1[r][k] * m2[k][c];
53             }
54         }
55     }
56
57     return res;
58 }

```

```

59
60 // Matrix multiplication mod p
61 vll matmulmod(const vll& m1, const vll& m2, ll MOD) {
62     ll nr = m1.size(), nc = m2[0].size();
63     ll maxk = m1.size(); // should be equal to m1[0].size()
64
65     vll res(nr, vll(nc, 0ll));
66
67
68     for(ll r = 0; r < nr; r++) {
69         for(ll k = 0; k < maxk; k++) {
70             for(ll c = 0; c < nc; c++) {
71                 res[r][c] = (res[r][k] + (m1[r][k] * m2[k][c]) % MOD) % MOD;
72             }
73         }
74     }
75
76     return res;
77 }
78
79 // Matrix-vector multiplication, no modulo
80 vll matvec(const vll& m, const vll& v) {
81     vll res;
82     ll res_size = m.size();
83     ll inp_size = v.size();
84
85     for(ll i = 0; i < res_size; i++) {
86         ll cur_sum = 0ll;
87         for(ll j = 0; j < inp_size; j++) {
88             cur_sum += v[j] * m[i][j];
89         }
90         res.push_back(cur_sum);
91     }
92
93     return res;
94 }
95
96 // Matrix-vector multiplication, with modulo
97 vll matvecmod(const vll& m, const vll& v, ll MOD) {
98     vll res;
99     ll res_size = m.size();
100     ll inp_size = v.size();
101
102     for(ll i = 0; i < res_size; i++) {
103         ll cur_sum = 0ll;
104         for(ll j = 0; j < inp_size; j++) {
105             cur_sum = (cur_sum + (v[j] * m[i][j]) % MOD) % MOD;
106         }
107         res.push_back(cur_sum);
108     }
109
110     return res;
111 }
112
113 // Creates an identity matrix of size n x n
114 vll id_mat(ll n) {
115     vll res;
116     for(ll i = 0; i < n; i++) {
117         vll res_row(n, 0ll);
118         res.push_back(res_row);
119     }
120     for(ll i = 0; i < n; i++) {
121         res[i][i] = 1ll;
122     }
123     return res;
124 }
125
126 // Fast matrix exponentiation
127 // m has to be a square matrix
128 // O(n^3 log pw)
129 vll matpowmod(const vll& m, ll pw, ll md) {
130     if(pw == 0) {
131         return id_mat(m.size());
132     }
133     vll sqr_sqrt = matpowmod(matmulmod(m, m, md), pw >> 1, md);
134     return (pw & 1 ? matmulmod(sqr_sqrt, m, md) : sqr_sqrt);
135 }
136
137 // Matrix Addition
138 vll matadd(const vll& m1, const vll& m2, ll md) {

```

```

139     ll r1 = m1.size(), c1 = m1[0ll].size();
140     ll r2 = m2.size(), c2 = m2[0ll].size();
141
142     assert(r1 == r2 && c1 == c2);
143
144     vll res(r1, vll(c1, 0ll));
145
146     for(ll i = 0ll; i < r1; i++) for(ll j = 0ll; j < c1; j++) res[i][j] = (m1[i][j] + m2[i][j]) % md;
147
148     return res;
149 }
150
151 // Matrix Subtraction
152 vll matsub(const vll& m1, const vll& m2, ll md) {
153     ll r1 = m1.size(), c1 = m1[0ll].size();
154     ll r2 = m2.size(), c2 = m2[0ll].size();
155
156     assert(r1 == r2 && c1 == c2);
157
158     vll res(r1, vll(c1, 0ll));
159
160     for(ll i = 0ll; i < r1; i++) for(ll j = 0ll; j < c1; j++) res[i][j] = ((m1[i][j] - m2[i][j]) % md + md) % md;
161
162     return res;
163 }
164
165 // Copy a matrix
166 vll matcopy(const vll& mat) {
167     ll r = mat.size(), c = mat[0ll].size();
168
169     vll res(r, vll(c, 0ll));
170
171     for(ll i = 0ll; i < r; i++) for(ll j = 0ll; j < c; j++) res[i][j] = mat[i][j];
172
173     return res;
174 }
175
176 // Matrix Transpose
177 vll transpose(const vll& mat) {
178     ll r = mat.size();
179
180     vll empty;
181     if(r == 0ll) return empty;
182
183     ll c = mat[0ll].size();
184
185     if(c == 0ll) return empty;
186
187     vll res(c, vll(r, 0ll));
188     for(ll i = 0ll; i < r; i++) for(ll j = 0ll; j < c; j++) res[j][i] = mat[i][j];
189
190     return res;
191 }
192
193 // In-place Gauss-Jordan Elimination mod p
194 // lc is the number of columns to reduce. useful for reducing augmented matrices
195 // returns the determinant
196 ll gj_inplace(vll& mat, ll lc, ll md) {
197     ll r = mat.size();
198
199     if(r == 0ll) return 0ll;
200     ll c = mat[0ll].size();
201     if(c == 0ll) return 0ll;
202
203     ll det = 1ll;
204
205     // reducing to row-echelon form
206     ll cc = 0ll;
207     for(ll cr = 0ll; cr < r && cc < min(c, lc); cr++) {
208         // find the first row in [i, r] that's nonzero
209         ll fstnonzero = -1ll;
210
211         do {
212             for(ll j = cr; j < r; j++) if(mat[j][cc] != 0ll) {fstnonzero = j; break;}
213
214             if(fstnonzero == -1ll) {
215                 cc++;
216                 det = 0ll;
217             }
218             while(fstnonzero == -1ll && cc < c);

```

```

219
220         if(cc >= c) break;
221
222         if(cr != fstnonzero) {
223             swap(mat[cr], mat[fstnonzero]);
224             det = (md - det);
225         }
226
227         // normalize the current row
228         const ll lead_inv = minv(mat[cr][cc], md);
229
230         det = (det * mat[cr][cc]) % md;
231         for(ll j = cc; j < c; j++) {
232             mat[cr][j] = (mat[cr][j] * lead_inv) % md;
233
234             mat[cr][j] %= md;
235             mat[cr][j] += md;
236             mat[cr][j] %= md;
237         }
238
239         // kill all terms below and above
240
241         for(ll j = 0ll; j < r; j++) {
242             if(j == cr) continue;
243             ll cur_lead_entry = mat[j][cc];
244             if(cur_lead_entry == 0ll) continue;
245
246             ll multi = (cur_lead_entry) % md;
247
248             for(ll i = cc; i < c; i++) {
249                 mat[j][i] -= (mat[cr][i] * multi) % md;
250                 mat[j][i] %= md;
251                 mat[j][i] += md;
252                 mat[j][i] %= md;
253             }
254         }
255
256         cc++;
257     }
258
259     return det;
260 }
261
262 // Gauss-Jordan Elimination mod p
263 vll gj(const vll& mat, ll lc, ll md) {
264     vll res = matcopy(mat);
265
266     gj_inplace(res, lc, md);
267
268     return res;
269 }
270
271 // Determinant Computation (Using Gauss-Jordan Elimination)
272 ll determinant(const vll& mat, ll md) {
273     vll matcpy = matcopy(mat);
274
275     return gj_inplace(matcpy, matcpy[0ll].size(), md);
276 }
277
278 // Row Space Basis Finder (i.e., nonzero row selector)
279 vll rowspace(const vll& mat, ll md) {
280     vll matcpy = matcopy(mat);
281
282     ll r = matcpy.size(), c = matcpy[0ll].size();
283     gj_inplace(matcpy, c, md);
284
285     vll res;
286
287
288     for(ll i = 0ll; i < r; i++) {
289         bool all_zero = true;
290
291         for(ll j = 0ll; j < c; j++) if(matcpy[i][j] != 0ll) {all_zero = false; break;}
292
293         if(!all_zero) res.push_back(matcpy[i]);
294     }
295
296     return res;
297 }
298

```

```

299 // Matrix Rank Finder (Note: rank(A) = dim(Row(A)) = dim(Col(A)))
300 ll matrank(const vvl& mat, ll md) {
301     if((ll)mat.size() == 0ll) return 0ll;
302     if((ll)mat[0ll].size() == 0ll) return 0ll;
303     vvl matcpy = matcopy(mat);
304
305     ll r = matcpy.size(), c = matcpy[0ll].size();
306     gj_inplace(matcpy, c, md);
307
308     ll res = 0ll;
309
310     for(ll i = 0ll; i < r; i++) {
311         bool all_zero = true;
312
313         for(ll j = 0ll; j < c; j++) if(matcpy[i][j] != 0ll) {all_zero = false; break;}
314
315         if(!all_zero) res++;
316     }
317     return res;
318 }
319
320 // Col Space Basis Finder (i.e., leading entry column selector)
321 // Note: Returns the column space as row vectors!
322 vvl colspace(const vvl& mat, ll md) {
323     vvl matcpy = matcopy(mat);
324
325     ll r = matcpy.size(), c = matcpy[0ll].size();
326
327     // reducing to row-echelon form
328     vvl res;
329
330     ll cc = 0ll;
331     for(ll cr = 0ll; cr < r && cc < c; cr++) {
332         // find the first row in [i, r] that's nonzero
333         ll fstnzero = -1ll;
334
335         do {
336             for(ll j = cr; j < r; j++) if(matcpy[j][cc] != 0ll) {fstnzero = j; break;}
337
338             if(fstnzero == -1ll) {
339                 cc++;
340             } while(fstnzero == -1ll && cc < c);
341
342             if(cc >= c) break;
343
344             if(cr != fstnzero) {
345                 swap(matcpy[cr], matcpy[fstnzero]);
346             }
347
348             // normalize the current row
349             const ll lead_inv = minv(matcpy[cr][cc], md);
350
351             for(ll j = cc; j < c; j++) {
352                 matcpy[cr][j] = (matcpy[cr][j] * lead_inv) % md;
353
354                 matcpy[cr][j] %= md;
355                 matcpy[cr][j] += md;
356                 matcpy[cr][j] %= md;
357             }
358
359             // kill all terms below and above
360
361             for(ll j = 0ll; j < r; j++) {
362                 if(j == cr) continue;
363                 ll cur_lead_entry = matcpy[j][cc];
364                 if(cur_lead_entry == 0ll) continue;
365
366                 ll multi = (cur_lead_entry) % md;
367
368                 for(ll i = cc; i < c; i++) {
369                     matcpy[j][i] -= (matcpy[cr][i] * multi) % md;
370                     matcpy[j][i] %= md;
371                     matcpy[j][i] += md;
372                     matcpy[j][i] %= md;
373                 }
374             }
375         }
376     }
377 }

```

```

379
380 // Adding the current column with a leading entry to the column basis
381 vvl next_vec(r, 0ll);
382
383 for(ll i = 0ll; i < r; i++) next_vec[i] = mat[i][cc];
384
385 res.push_back(next_vec);
386
387 cc++;
388 }
389
390 return res;
391 }
392
393 // Null Space Basis Finder
394 vvl nullspace(const vvl& mat, ll md) {
395     vvl matcpy = matcopy(mat);
396     gj_inplace(matcpy, matcpy[0ll].size(), md);
397
398     // Find the indices of the free parameters
399     ll r = matcpy.size(), c = matcpy[0ll].size();
400     vvl leads;
401     vvl freparams;
402     set<ll> freparamsset;
403
404     ll cur_xpos = 0ll;
405     for(ll i = 0ll; i < r; i++) {
406         while(cur_xpos < c && matcpy[i][cur_xpos] == 0ll) {
407             cur_xpos++;
408         }
409
410         if(cur_xpos == c) break;
411
412         leads.push_back(cur_xpos);
413     }
414
415     ll cur_ind = 0ll;
416
417     ll num_leads = leads.size();
418
419     for(ll i = 0ll; i < c; i++) {
420         if(cur_ind < num_leads && i == leads[cur_ind]) {
421             cur_ind++;
422         } else {
423             freparams.push_back(i);
424             freparamsset.insert(i);
425         }
426     }
427
428     // For each free parameter, put a default value into the corresponding vector
429     ll num_free_params = freparams.size();
430
431     if(num_free_params == 0ll) {
432         vvl empty;
433         return empty;
434     }
435
436     vvl res(num_free_params, vll(c, 0ll));
437
438     for(ll i = 0ll; i < num_free_params; i++) {
439         res[i][freparams[i]] = 1ll;
440     }
441
442     // Process each leading entry
443     cur_xpos = 0ll;
444     for(ll i = 0ll; i < r; i++) {
445         while(cur_xpos < c && matcpy[i][cur_xpos] == 0ll) {
446             cur_xpos++;
447         }
448
449         if(cur_xpos == c) break;
450
451         ll cur_free_param_ind = 0ll;
452         for(ll j = 0ll; j < c && cur_free_param_ind < num_free_params; j++) {
453             if((ll)(freparamsset.count(j)) > 0ll) {
454                 if(j > cur_xpos) {
455                     res[cur_free_param_ind][cur_xpos] = (md - matcpy[i][j]) % md;
456                 }
457             }
458             cur_free_param_ind++;

```

```

459     }
460     }
461 }
462
463 return res;
464 }
465
466 // Matrix Inverter
467 vvl matinv(const vvl& mat, ll md) {
468     ll r = mat.size(), c = mat[0ll].size();
469     vvl augmented(r, vll(c << 1ll, 0ll));
470
471     for(ll i = 0ll; i < r; i++) for(ll j = 0ll; j < c; j++) augmented[i][j] = mat[i][j];
472
473     for(ll i = 0ll; i < r; i++) augmented[i][i + c] = 1ll;
474
475     gj_inplace(augmented, c, md);
476
477     vvl res(r, vll(c, 0ll));
478
479     for(ll i = 0ll; i < r; i++) for(ll j = 0ll; j < c; j++) res[i][j] = augmented[i][j + c];
480
481     return res;
482 }
483
484 // Eigenvector Finder given Eigenvalue
485 // (Could be used to find the stationary distribution of a markov chain)
486 vvl find_eigenvectors(const vvl& mat, ll lambda, ll md) {
487     vvl matcpy = matcopy(mat);
488
489     ll r = matcpy.size();
490     assert(r > 0ll);
491     ll c = matcpy[0ll].size();
492     assert(c > 0ll);
493
494     for(ll i = 0ll; i < r; i++) {
495         matcpy[i][i] -= lambda;
496         matcpy[i][i] %= md;
497         matcpy[i][i] += md;
498         matcpy[i][i] %= md;
499     }
500
501     vvl eigenvecs = nullspace(matcpy, md);
502
503     return eigenvecs;
504 }
505
506 // Find stationary distribution of a markov chain
507 vvl find_stationary_distrib(const vvl& tr_mat, ll md) {
508     vvl vecs = find_eigenvectors(transpose(tr_mat), 1ll, md);
509
510     ll numvecs = vecs.size();
511
512     if(numvecs == 0ll) {
513         vvl empty;
514         return empty;
515     }
516
517     ll vecdim = vecs[0ll].size();
518
519     for(ll i = 0ll; i < numvecs; i++) {
520         ll sum_entries = 0ll;
521
522         for(ll j = 0ll; j < vecdim; j++) {
523             sum_entries += vecs[i][j];
524             sum_entries %= md;
525             sum_entries += md;
526             sum_entries %= md;
527         }
528
529         for(ll j = 0ll; j < vecdim; j++) {
530             vecs[i][j] *= minv(sum_entries, md);
531             vecs[i][j] %= md;
532             vecs[i][j] += md;
533             vecs[i][j] %= md;
534         }
535     }
536
537     return vecs;
538 }

```

```

539
540 // Check basis b1 is subset of b2
541 bool is_basis_subset(const vll& b1, const vll& b2, ll md) {
542     ll lc = b2.size();
543     ll bs = b1.size();
544
545     vll concat = b2;
546     for(ll i = 0ll; i < bs; i++) concat.push_back(b1[i]);
547
548     vll reducedmat = gj(transpose(concat), lc, md);
549
550     ll nr = reducedmat.size();
551
552     for(ll i = 0ll; i < nr; i++) {
553         bool all_left_zeros = true;
554         bool all_right_zeros = true;
555
556         for(ll j = 0ll; j < lc; j++) {
557             if(reducedmat[i][j] != 0ll) {
558                 all_left_zeros = false;
559                 break;
560             }
561         }
562
563         for(ll j = lc; j < lc + bs; j++) {
564             if(reducedmat[i][j] != 0ll) {
565                 all_right_zeros = false;
566                 break;
567             }
568         }
569
570         // if left does not imply right, return false (formal negation of implication)
571         if(all_left_zeros && !all_right_zeros) return false;
572     }
573
574     return true;
575 }
576
577 // Check if Two Bases are Equal
578 bool is_basis_eq(const vll& b1, const vll& b2, ll md) {
579     return is_basis_subset(b1, b2, md) && is_basis_subset(b2, b1, md);
580 }
581
582 // Check if Vector in Span (equivalent to checking if the span of the singular vector is a subset of
583 // the set of vectors)
584 bool is_in_span(const vll& v, const vll& b, ll md) {
585     vll b1;
586     b1.push_back(v);
587
588     return is_basis_subset(b1, b, md);
589 }

```

13.8. Hyperbola Techniques

13.8.1. Mobius Function

Denoted as $\mu(n)$, $\mu : \mathbb{N} \rightarrow \{-1, 0, 1\}$. It is defined as 0 if n is not squarefree, 1 if it is squarefree and has an even number of prime divisors, and -1 if it is squarefree with an odd number of prime divisors.

```

1 // Precomputes the Mobius Function
2 vll precomp_mobius(ll n) {
3     ll MAX = n;
4     vb is_sqfree(MAX + 1ll, true);
5
6     for(ll i = 2ll; i * i <= MAX; i++) {
7         for(ll k = i * i; k <= MAX; k += i * i) {
8             is_sqfree[k] = false;
9         }
10    }
11
12    vll facts;
13
14    for(ll i = 0ll; i <= MAX; i++) {
15        vll facts;
16        facts.push_back(facts);
17    }
18

```

```

19    for(ll i = 2ll; i <= MAX; i++) {
20        if((ll)facts[i].size() == 0ll) {
21            // Is a prime
22            for(ll k = i; k <= MAX; k += i) {
23                facts[k].push_back(i);
24            }
25        }
26    }
27
28    vll mobius(MAX + 1ll, 0ll);
29
30    for(ll i = 1ll; i <= MAX; i++) {
31        if(is_sqfree[i]) {
32            mobius[i] = (((ll)(facts[i].size()) & 0b1ll) == 0ll ? 1ll : -1ll);
33        }
34    }
35
36    return mobius;
37 }

```

13.8.2. Mobius Inversion

A useful formula for computing sums of functions evaluated at divisors:

$$f(n) = \sum_{g \mid n} h(g) \leftrightarrow h(n) = \sum_{g \mid n} \mu(n) f\left(\frac{n}{g}\right)$$

13.8.3. Hyperbola Technique: Normal Form

Consider sums of the form $f(n) = \sum_{1 \leq x \leq n} g(x) h\left(\left\lfloor \frac{n}{x} \right\rfloor\right)$.

We observe that the size of $\langle n \rangle = \left\{ \left\lfloor \frac{n}{x} \right\rfloor \mid x \in [1, n] \cap \mathbb{Z} \right\}$ is $O(\sqrt{n})$.

Furthermore, $\left\lfloor \frac{n}{x} \right\rfloor = v \leftrightarrow \left\lfloor \frac{n}{v+1} \right\rfloor < x \leq \left\lfloor \frac{n}{v} \right\rfloor$. Thus, we can compute the sum above in $O\left(n^{\frac{1}{2}}\right)$ time using prefix sums:

$$f(n) = \sum_{v \in \langle n \rangle} h(v) \left(G\left(\left\lfloor \frac{n}{v} \right\rfloor\right) - G\left(\left\lfloor \frac{n}{v+1} \right\rfloor\right) \right)$$

In general, it is possible to compute $f(n) = \sum_{1 \leq x \leq n} g(x) h\left(\left\lfloor \frac{n}{x} \right\rfloor\right)$ in $O\left(n^{\frac{1}{k+1}}\right)$ time, assuming $G(x)$ can be computed in $O(1)$. This is because the set $\langle n \rangle_k = \left\{ \left\lfloor \frac{n}{x^k} \right\rfloor \mid x \in [1, n] \cap \mathbb{Z} \right\}$ has $O\left(n^{\frac{1}{k+1}}\right)$ elements.

```

1 inline ll f(ll n) {
2     return n;
3 }
4
5 inline ll g(ll n) {
6     return n;
7 }
8
9 ll hypertech(ll n) {
10    ll x = 1ll;
11    ll v = n;
12    ll ans = 0ll;
13
14    while(x <= n) {
15        ans += g(v) * (f(n / v) - f(n / (v + 1ll)));
16        x += n / v - n / (v + 1ll);
17        v = n / x;
18    }
19
20    return ans;
21 }

```

13.8.4. Hyperbola Technique: Inverse Form

Now, the roles are swapped. We know f but do not know h in the following equation:

$$f(n) = \sum_{1 \leq x \leq n} h\left(\left\lfloor \frac{n}{x} \right\rfloor\right)$$

This occurs when, for instance, we try to compute the number of lattice points in the first quadrant such that $\max(x, y) \leq n$ that are visible from the origin (i.e., no other points lie on the line segment between this point and the origin). The number of visible lattice points is given by:

$$h(n) = \sum_{1 \leq x \leq n} \sum_{1 \leq y \leq n, \gcd(x, y)=1} 1$$

Let $f(n) = \sum_{1 \leq x \leq n} \sum_{1 \leq y \leq n} 1 = n^2$. Thus:

$$\begin{aligned}
 f(n) &= \sum_{1 \leq g \leq n} \sum_{1 \leq x \leq n} \sum_{1 \leq y \leq n, \gcd(x, y)=g} 1 \\
 &= \sum_{1 \leq g \leq n} \sum_{1 \leq gx' \leq n} \sum_{1 \leq gy' \leq n, \gcd(x', y')=g} 1 \\
 &= \sum_{1 \leq g \leq n} \sum_{1 \leq x' \leq \frac{n}{g}} \sum_{1 \leq y' \leq \frac{n}{g}, \gcd(x', y')=1} 1 \\
 &= \sum_{1 \leq g \leq n} h\left(\frac{n}{g}\right)
 \end{aligned}$$

We can compute h in $O\left(n^{\frac{3}{4}}\right)$ through a DP-like process accelerated with the normal-form hyperbola technique. Note that the values of h required are exactly those in $\langle n \rangle$.

$$h(n) = f(n) - \sum_{2 \leq g \leq n} h\left(\frac{n}{g}\right)$$

We can do better using a sieve technique (can't implement this rn): $O\left(n^{\frac{2}{3}} (\log(n))^{\frac{1}{3}}\right)$

The implementation is shown in the next section.

13.8.5. Mertens Function

One can use the identity $1(n) = \sum_{1 \leq g \leq n} M\left(\frac{n}{g}\right)$ to compute the Mertens Function, the partial sum of the Mobius Function.

This allows us to compute f given h for the following case, as follows:

$$h(n) = \sum_{1 \leq x \leq n} \mu(x) f\left(\frac{n}{x}\right) \leftrightarrow f(n) = \sum_{v \in \langle n \rangle} h(v) \left[M\left(\left\lfloor \frac{n}{v} \right\rfloor\right) - M\left(\left\lfloor \frac{n}{v+1} \right\rfloor\right) \right]$$

Implementation is shown below:

```

1 mll mertens(ll n) {
2     ll x = n;
3     ll v = 1ll;
4     ll ans = 0ll;
5
6     // note: usage of map introduces extra log factors
7     // but simplifies the implementation
8     mll res;
9
10    while(x > 0ll) {
11        v = n / x;
12        res[v] = 1ll;
13        ll x2 = 1ll, v2 = v;
14        x2 += v / v2 - v / (v2 + 1ll);
15        v2 = v / x2;
16        while(v2 > 0ll) {
17            res[v] -= res[v2] * (v / v2 - v / (v2 + 1ll));
18            x2 += v / v2 - v / (v2 + 1ll);
19            v2 = v / x2;
20        }
21        x -= n / v - n / (v + 1ll);

```

```
22 }
23
24     return res;
25 }
26
27 // By default, this computes the number of lattice points visible from the origin
28 inline ll h(ll n) {
29     return n * n;
30 }
31
32 /*
33 suppose h(n) = sum_{1 <= g <= n} f(n / g)
34
35 and h is given
36
37 hypertech_inv finds f
38 */
39 ll hypertech_inv(ll n) {
40     mll mertensvals = mertens(n);
41
42     ll x = 1ll;
43     ll v = n;
44     ll ans = 0ll;
45
46     while(x <= n) {
47         ans += h(v) * (mertensvals[n / v] - mertensvals[n / (v + 1ll)]);
48         x += n / v - n / (v + 1ll);
49         v = n / x;
50     }
51
52     return ans;
53 }
```

```
30 Point p1 = a[0], p2 = a.back();
31 vector<Point> up, down;
32
33 up.P(p1);
34 down.P(p1);
35
36 for (int i = 1; i < S(a); i++) {
37     if (i == S(a) - 1 || cw(p1, a[i], p2, include_collinear)) {
38         while (S(up) >= 2
39             && !cw(up[S(up) - 2], up[S(up) - 1], a[i], include_collinear))
40             up.pop_back();
41         up.P(a[i]);
42     }
43     if (i == S(a) - 1 || ccw(p1, a[i], p2, include_collinear)) {
44         while (S(down) >= 2
45             && !ccw(
46                 down[S(down) - 2], down[S(down) - 1], a[i], include_collinear))
47             down.pop_back();
48         down.P(a[i]);
49     }
50 }
51
52 if (include_collinear && S(up) == S(a)) {
53     reverse(A(a));
54     return;
55 }
56
57 a.clear();
58
59 for (int i = 0; i < S(up); i++) a.P(up[i]);
60 for (int i = S(down) - 2; i > 0; i--) a.P(down[i]);
61 }
```

```
16
17 Vec2 operator/(T o) const {
18     return {x / o, y / o};
19 }
20
21 bool operator==(const Vec2& o) const {
22     return x == o.x && y == o.y;
23 }
24
25 Vec2 operator-() const {
26     return {-x, -y};
27 }
28
29 T norm() const {
30     return x * x + y * y;
31 }
32 };
33 template <typename T>
34 using Line = Vec2<T>;
35
36 template <typename T>
37 using vl = vector<Line<T>>;
38
39 // Computes the dot product of two vectors
40 template <typename T>
41 inline T dot(const Vec2<T>& a, const Vec2<T>& b) {
42     return a.x * b.x + a.y * b.y;
43 }
44
45 // Computes the "2d cross product" of two vectors
46 template <typename T>
47 inline T cross(const Vec2<T>& a, const Vec2<T>& b) {
48     return a.x * b.y - a.y * b.x;
49 }
50
51 // Checks if three points are collinear
52 template <typename T>
53 inline bool collinear(const Vec2<T>& a, const Vec2<T>& b, const Vec2<T>& c) {
54     return cross(b - a, c - a) == (T)0ll;
55 }
56
57 // Checks if two vectors are collinear
58 template <typename T>
59 inline bool collinear(const Vec2<T>& a, const Vec2<T>& b) {
60     return cross(a, b) == (T)0ll;
61 }
62
63 // Checks if point c lies on the line segment spanned by a and b
64 template <typename T>
65 inline bool c_in_ab(const Vec2<T>& a, const Vec2<T>& b, const Vec2<T>& c) {
66     return collinear(a, b, c) && dot(a - c, b - c) < (T)0ll;
67 }
68
69 // Checks if v is STRICTLY counterclockwise u
70 template <typename T>
71 inline bool is_ccw(const Vec2<T>& u, const Vec2<T>& v) {
72     return cross(u, v) > (T)0ll;
73 }
74
75 // Checks if v is STRICTLY clockwise u
76 template <typename T>
77 inline bool is_cw(const Vec2<T>& u, const Vec2<T>& v) {
78     return cross(u, v) < (T)0ll;
79 }
80
81 // From the vantage point of point a, is point v ccw of point u?
82 template <typename T>
83 inline bool is_ccw(const Vec2<T>& a, const Vec2<T>& u, const Vec2<T>& v) {
84     return is_ccw(u - a, v - a);
85 }
86
87 // Check if two vectors are perpendicular
88 template <typename T>
89 inline bool is_perp(const Vec2<T>& a, const Vec2<T>& b) {
90     return dot(a, b) == (T)0ll;
91 }
92
93 // Projects u onto v
94 Vec2<double> project(const Vec2<double>& u, const Vec2<double>& v) {
95     return v * (dot(u, v) / v.norm());
96 }
```

13.9. Geometry

13.9.1. Convex Hull

The convex hull is the *smallest convex polygon* that contains all the given points. This implementation uses the monotone chain algorithm, i.e., find the leftmost and rightmost points. If multiple points are found, take the lowest of the left and the highest of the right. Draw a line connecting the two points to split it into two sets, before constructing an upper and lower hull.

You need to remember to remove duplicate points in the input! Because this implementation yields the points in clockwise order, you need to reverse the output to get it in anticlockwise order.

```
1 struct Point {
2     ll x, y;
3 };
4
5 int orientation(Point a, Point b, Point c) {
6     ll v = a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y);
7     if (v < 0) return -1; // clockwise
8     if (v > 0) return +1; // counter-clockwise
9     return 0;
10 }
11
12 bool cw(Point a, Point b, Point c, bool include_collinear) {
13     int o = orientation(a, b, c);
14     return o < 0 || (include_collinear && o == 0);
15 }
16
17 bool ccw(Point a, Point b, Point c, bool include_collinear) {
18     int o = orientation(a, b, c);
19     return o > 0 || (include_collinear && o == 0);
20 }
21
22 // WARN: returns the hull in CLOCKWISE order (affects `a` inplace)
23 void convex_hull(vector<Point> &a, bool include_collinear = false) {
24     if (S(a) == 1) return;
25
26     sort(A(a), [](Point a, Point b) {
27         return make_pair(a.x, a.y) < make_pair(b.x, b.y);
28     });
29 }
```

13.9.2. Various Geometry Tools

The tools include ways to manipulate and process vectors, such as vector addition, subtraction, dot products, cross products, projection, reflection, rotation, collinear checks, perpendicularity checks, counterclockwise and clockwise checks, and angle computations.

It also includes some basic tools to handle polygons with lattice point coordinates, such as area computations, boundary point counters, and interior point counters.

Some useful formulas:

$$\text{Ref}_{\vec{u}}^{-1}(\vec{x}) = \langle \vec{u} \cdot \vec{x}, \vec{u} \times \vec{x} \rangle$$

45-degree Rotation (CCW) with scaling: $(x, y) \rightarrow (x - y, x + y)$

45-degree Rotation (CW) with scaling: $(x, y) \rightarrow (x + y, x - y)$

Pick's Theorem: $A = I + \frac{B}{2} - 1$

In the future, try adding line/line intersections, line/circle intersections, subspace intersections, and the like.

```
1 template <typename T>
2 struct Vec2 {
3     T x, y;
4
5     Vec2 operator+(const Vec2& o) const {
6         return {x + o.x, y + o.y};
7     }
8
9     Vec2 operator-(const Vec2& o) const {
10        return {x - o.x, y - o.y};
11    }
12
13    Vec2 operator*(T o) const {
14        return {x * o, y * o};
15    }
16 }
```

14. Strings

14.1. String Splitting

Because C++ doesn't offer a split function by default, we may have to write our own sometimes.

```
1 auto string_split = [](const string &s, const string &delims) {
2     size_t prv = 0, pos;
3     vector<string> parsed;
4     while ((pos = s.find_first_of(delims, prv)) != string::npos) {
5         if (pos > prv) parsed.push_back(s.substr(prv, pos - prv));
6         prv = pos + 1;
7     }
8     if (prv < size(s)) parsed.push_back(s.substr(prv, string::npos));
9     return parsed;
10 };
```

14.2. Hashing

You may use two different prime numbers to create a stronger hash.

```
1 int NMAX = 500'000; // WARN: rmb to tweak
2 vector<Mint> pwr(NMAX), invs(NMAX);
3
4 class Shash {
5     vector<Mint> p;
6
7 public:
8     static constexpr Mint pwr = 31;
9     static constexpr Mint inv = pwr.i();
10    Shash(string s) {
11        p.resize(S(s));
12        for (int i = 0; i < S(s); i++) {
13            p[i] = pwr[s[i]] * (s[i] - 'a' + 1);
14        }
15        partial_sum(begin(p), end(p), begin(p));
16    }
17    Mint query(int l, int r) {
18        if (l == 0) return p[r];
19        return (p[r] - p[l - 1]) * invs[l];
20    }
21 };
22
23 int main() {
24     Mint pw = 1, in = 1;
25     for (int i = 0; i < NMAX; i++) {
26         pwr[i] = pw, invs[i] = in;
27         pw *= Shash::pwr, in *= Shash::inv;
28     }
29 }
```

14.3. Manacher's Algorithm

Computes the maximum length of a palindrome centered at any index.

```
1 class Manacher {
2     deque<int> p;
3     deque<int> manacher_odd(string s) {
4         int n = s.size();
5         s = '{' + s + '}';
6         deque<int> p(n + 2);
7         int l = 1, r = 1;
8         for (int i = 1; i <= n; i++) {
9             p[i] = max(0, min(r - i, p[l + (r - i)]));
10            while (s[i - p[i]] == s[i + p[i]]) p[i]++;
11            if (i + p[i] > r) l = i - p[i], r = i + p[i];
12        }
13        p.pop_front(), p.pop_back();
14        return p;
15    }
16
17 public:
18     Manacher(const string &s) {
19         string t(1, '|');
20         for (auto &c : s) t += c, t += '|';
21         auto ans = manacher_odd(t);
```

```
22     ans.pop_front(), ans.pop_back();
23     p = ans;
24 }
25 int get_len(int i, bool is_odd) {
26     // NOTE: center palindrome at 'i' and 'i+1' if 'is_odd' is false
27     return is_odd ? 2 * (p[2 * i] / 2 - 1) + 1 : p[2 * i + 1] - 1;
28 }
29 };
```

14.4. Z-function

The *Z-function* of a string is an array of length n where the i -th element is equal to the greatest number of characters starting from position i that coincides with the first characters of the string.

```
1 auto z_function = [](const string &s) -> vector<int> {
2     int l = 0, r = 0, n = s.size();
3     vector<int> z(n);
4     for (int i = 1; i < n; i++) {
5         if (i < r) z[i] = min(r - i, z[i - l]);
6         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
7         if (i + z[i] > r) l = i, r = i + z[i];
8     }
9     return z;
10 };
```

14.5. Trie

```
1 constexpr int K = 26;
2
3 struct Vertex {
4     vector<int> nxt;
5     bool is_end;
6     int cnt;
7     Vertex()
8         : nxt(K, -1)
9         , is_end(false)
10        , cnt(0) {}
11 };
12
13 vector<Vertex> trie(1);
14
15 void insert(const string &s) {
16     int u = 0;
17     for (const auto &c : s) {
18         int i = c - 'a';
19         if (trie[u].nxt[i] == -1) {
20             trie[u].nxt[i] = s.size(trie);
21             trie.emplace_back();
22         }
23         u = trie[u].nxt[i];
24         trie[u].cnt++;
25     }
26     trie[u].is_end = true;
27 }
28
29 ll search(const string &s) {
30     int u = 0;
31     ll ans = 0;
32     for (const auto &c : s) {
33         int i = c - 'a';
34         if (trie[u].nxt[i] == -1) break;
35         u = trie[u].nxt[i];
36         ans += trie[u].cnt;
37     }
38     return ans;
39 }
```

You can also find the maximum XOR of a subarray.

```
1 class Trie {
2     int max_length;
3     int node_cnt;
4     vector<int> stop;
5     vector<vector<int>> trie;
6 }
```

```
96 }
97
98 // Reflects x across v
99 Vec2<double> refl(const Vec2<double>& x, const Vec2<double>& v) {
100     return v + (dot(x, v) * 2.0 / v.norm()) - x;
101 }
102
103 // Rotates (and scales) by another vector in such a way that the resultant vector is still a lattice
104 // point
105 // Specifically, rotates x by the direction specified by u (i.e., the angle u makes with the +x-axis)
106 // w.r.t. the origin
107 template<typename T>
108 Vec2<T> rotscale(const Vec2<T>& x, const Vec2<T>& u) {
109     return {u.x * x.x - u.y * x.y, u.y * x.x + u.x * x.y};
110 }
111
112 // Computes the cosine of the angle between two vectors
113 double vec_cosine(const Vec2<double>& u, const Vec2<double>& v) {
114     return dot(u, v) / sqrt(u.norm() * v.norm());
115 }
116
117 // Computes the counterclockwise radian measure spanned from u to v
118 double angle(const Vec2<double>& u, const Vec2<double>& v) {
119     double rawtheta = acos(min(1.0, max(-1.0, vec_cosine(u, v))));
120
121     return rawtheta + (is_cw(u, v) ? M_PI : 0.0);
122 }
123
124 // Computes TWICE the area of a polygon
125 // Uses the Shoelace Formula
126 template<typename T>
127 using Polygon = vector<Vec2<T>>;
128
129 ll twice_area(const Polygon<ll>& poly) {
130     ll n = poly.size();
131
132     ll ans = 0ll;
133     for (ll i = 0ll; i < n - 1ll; i++) {
134         ans += poly[i].x * poly[i + 1ll].y - poly[i].y * poly[i + 1ll].x;
135     }
136     ans += poly[n - 1ll].x * poly[0ll].y - poly[n - 1ll].y * poly[0ll].x;
137
138     return abs(ans);
139 }
140
141 // Computes the number of lattice points from (0, 0) to (x, y)
142 ll num_lattice_pts_hit(const Vec2<ll>& v) {
143     ll avx = abs(v.x), avy = abs(v.y);
144     if (avx == 0ll && avy == 0ll) return 1ll;
145     return gcd(avx, avy) + 1ll;
146 }
147
148 // Computes the number of points on the boundary of a polygon with integer coordinates
149 ll num_lattice_pts_hit(const Polygon<ll>& poly) {
150     ll n = poly.size();
151     ll ans = 0ll;
152
153     for (ll i = 0ll; i < n - 1ll; i++) {
154         ans += num_lattice_pts_hit(poly[i + 1ll] - poly[i]) - 1ll;
155     }
156
157     ans += num_lattice_pts_hit(poly[0ll] - poly[n - 1ll]) - 1ll;
158
159     return ans;
160 }
161
162 // Computes the number of points on the interior of a polygon with integer coordinates
163 // Uses Pick's Theorem: A = i + b / 2 - 1 -> 2 i = 2 A - b + 2
164 ll num_interior_pts(const Polygon<ll>& poly) {
165     ll num_boundary = num_lattice_pts_hit(poly);
166     ll twice_a = twice_area(poly);
167
168     return (twice_a - num_boundary + 2ll) >> 1ll;
169 }
```

```

7 public:
8     Trie(int max_length)
9         : max_length(max_length)
10         , node_cnt(0)
11         , stop(max_length + 1, false)
12         , trie(max_length, vector<int>(2)) {}
13 void insert(int word) {
14     int node = 0;
15     for (int i = 30; i >= 0; i--) {
16         int b = (word >> i) & 1;
17         if (!trie[node][b]) trie[node][b] = ++node_cnt;
18         node = trie[node][b];
19     }
20     stop[node] = true;
21 }
22 int find(int word) {
23     int node = 0, s = 0;
24     for (int i = 30; i >= 0; i--) {
25         int b = (word >> i) & 1;
26         if (trie[node][b ^ 1]) {
27             if (b ^ 1) s |= (1 << i);
28             node = trie[node][b ^ 1];
29         } else {
30             if (b) s |= (1 << i);
31             node = trie[node][b];
32         }
33     }
34     return s;
35 }
36 };

```

```

44
45 int n;
46 string s;
47 auto sar = SuffixArray::build(s);
48
49 auto compare = [](const string &t, int suf_idx) -> int {
50     for (int i = 0; i < min((int)s.size(t), n - suf_idx); i++) {
51         if (t[i] < s[suf_idx + i]) return -1;
52         if (t[i] > s[suf_idx + i]) return 1;
53         if (i == s.size(t) - 1) return 0;
54     }
55     return 1;
56 };
57
58 auto count_substr = [](const string &t) -> int {
59     // WARN: `sar` := suffix array (declared in the outer scope)
60     int lo = 0, hi = n - 1, mi, an = -1, na = -1;
61     while (lo <= hi) {
62         mi = midpoint(lo, hi);
63         if (compare(t, sar[mi]) > 0) an = mi, lo = mi + 1;
64         else hi = mi - 1;
65     }
66     lo = 0, hi = n - 1;
67     while (lo <= hi) {
68         mi = midpoint(lo, hi);
69         if (compare(t, sar[mi]) >= 0) na = mi, lo = mi + 1;
70         else hi = mi - 1;
71     }
72     return na - an;
73 };

```

```

46     ans += (1 << k);
47     i += (1 << k);
48     j += (1 << k);
49 }
50 }
51 return ans;
52 }
53 static vector<int> build(const string &_) { // NOTE: call at the start
54     string s = _ + '$';
55     vector<int> a = sort_cyclic_shifts(s);
56     a.erase(begin(a));
57     return a;
58 }
59 };
60 int SuffixArray::n;
61 vector<vector<int>> SuffixArray::cmat;

```

14.7. Aho-Corasick Algorithm

This variant checks if any substring of a string `s` matches any string in the dictionary.

```

1 const int NMAX = 1'000 * 100; // WARN: rmb to tweak
2
3 int trie[NMAX + 1][26], fail[NMAX + 1], stop[NMAX + 1], idx;
4
5 void insert(const string &s) {
6     int i = 0;
7     for (auto &c : s) {
8         if (!trie[i][c - 'a']) trie[i][c - 'a'] = ++idx;
9         i = trie[i][c - 'a'];
10    }
11    stop[i] = true;
12 }
13 void build() { // NOTE: construct suffix link(s)
14     queue<int> q;
15     for (int i = 0; i < 26; i++)
16         if (trie[0][i]) q.push(trie[0][i]);
17     while (!empty(q)) {
18         int u = q.front();
19         q.pop();
20         stop[u] |= stop[fail[u]];
21         for (int i = 0; i < 26; i++) {
22             if (trie[u][i]) {
23                 fail[trie[u][i]] = trie[fail[u]][i];
24                 q.push(trie[u][i]);
25             } else {
26                 trie[u][i] = trie[fail[u]][i];
27             }
28         }
29     }
30 }
31 bool search(const string &s) {
32     int i = 0;
33     for (auto &c : s) {
34         i = trie[i][c - 'a'];
35         if (stop[i]) return true;
36     }
37     return false;
38 }

```

14.6. Suffix Array

A *suffix array* contains integers that represent the *starting indexes* of all suffixes of a given string, in sorted order.

```

1 class SuffixArray {
2     static constexpr int alphabet = 256;
3     static vector<int> sort_cyclic_shifts(const string &s) {
4         int n = s.size(), classes = 1;
5         vector<int> p(n), c(n), pn(n), cn(n), cnt(max(alphabet, n), 0);
6         for (int i = 0; i < n; i++) cnt[s[i]]++;
7         for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
8         for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
9         c[p[0]] = 0;
10        for (int i = 1; i < n; i++) {
11            if (s[p[i]] != s[p[i - 1]]) classes++;
12            c[p[i]] = classes - 1;
13        }
14        for (int h = 0; (1 << h) < n; h++) {
15            for (int i = 0; i < n; i++) {
16                pn[i] = p[i] - (1 << h);
17                if (pn[i] < 0) pn[i] += n;
18            }
19            fill(begin(cnt), begin(cnt) + classes, 0);
20            for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
21            for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
22            for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
23            cn[p[0]] = 0;
24            classes = 1;
25            for (int i = 1; i < n; i++) {
26                pair<int, int> cur = { c[p[i]], c[(p[i] + (1 << h)) % n] };
27                pair<int, int> prv = { c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n] };
28                if (cur != prv) classes++;
29                cn[p[i]] = classes - 1;
30            }
31            c.swap(cn);
32        }
33        return p;
34    }
35
36 public:
37     static vector<int> build(const string &_) {
38         string s = _ + '$';
39         vector<int> a = sort_cyclic_shifts(s);
40         a.erase(begin(a));
41         return a;
42     }
43 };

```

You can also modify it to retrieve the length of the longest common prefix between two suffixes starting at `i` and `j`.

```

1 class SuffixArray {
2     static constexpr int alphabet = 256;
3
4 public:
5     static int n;
6     static vector<vector<int>> cmat;
7     static vector<int> sort_cyclic_shifts(const string &s) {
8         n = s.size();
9         int classes = 1;
10        vector<int> p(n), c(n), pn(n), cn(n), cnt(max(alphabet, n), 0);
11        for (int i = 0; i < n; i++) cnt[s[i]]++;
12        for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
13        for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
14        c[p[0]] = 0;
15        for (int i = 1; i < n; i++) {
16            if (s[p[i]] != s[p[i - 1]]) classes++;
17            c[p[i]] = classes - 1;
18        }
19        cmat.P(c);
20        for (int h = 0; (1 << h) < n; h++) {
21            for (int i = 0; i < n; i++) {
22                pn[i] = p[i] - (1 << h);
23                if (pn[i] < 0) pn[i] += n;
24            }
25            fill(begin(cnt), begin(cnt) + classes, 0);
26            for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
27            for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
28            for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
29            cn[p[0]] = 0;
30            classes = 1;
31            for (int i = 1; i < n; i++) {
32                pair<int, int> cur = { c[p[i]], c[(p[i] + (1 << h)) % n] };
33                pair<int, int> prv = { c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n] };
34                if (cur != prv) classes++;
35                cn[p[i]] = classes - 1;
36            }
37            c.swap(cn);
38            cmat.P(c);
39        }
40        return p;
41    }
42    static int get_lcp(int i, int j) { // WARN: doesn't work for `i == j`
43        int ans = 0;
44        for (int k = __lg(n); k >= 0; k--) {
45            if (cmat[k][i % n] == cmat[k][j % n]) {

```